Open Scholarship Toolkit

Social Determinants of Health Data, Reproducible Analysis, and Open Sharing

Brian Chen

Table of Contents

Part 1. Data Orientation 1
1. Social Determinants of Health (SDOH) as Research Variables 1
2. The Five Healthy People 2030 SDOH Domains 2
3. The 10 Dataset Entries (8 Distinct Sources) 3
4. How To Use This Toolkit 5
5. Open Scholarship Requirements 5
6. Part 1 Checklist 8

Part 2. Download & R Workflow 1
1. R Environment Setup 1
2. Data Download Scripts (Scripts #1–#5) 3
3. Merge All Datasets into Master CSV (Script 6) 18
4. Statistical Analyses 19
5. Export and Data Dictionary 40
6. Part 2 Checklist 44

Part 3. Tableau, Repository & Open Sharing 1
1. Connect Your Data to Tableau 1
2. Choose Your Map Type 2
3. Asset Maps 4
4. Thematic Maps 8
5. Assemble Your Dashboard 14
6. Build Your Story Map 17
7. Adapt and Share 19
8. Make Your Project Citable, Reproducible, and Findable 21
9. Part 3 Checklists 24

Part 1. Data Orientation

What social determinants of health are, how Healthy People 2030 organizes them, which 8 unique datasets this toolkit uses across 10 entries, and how to get started.

Audience: Undergraduate and MPH students | **Level:** Beginner | **Time: 1** hour

Learning Objectives

- Define social determinants of health (**SDOH**) using Healthy People 2030 (**HP 2030**).
- Identify the five HP 2030 **SDOH domains** and provide at least two examples of variables for each.
- Name the 8 datasets (10 table entries) in this toolkit and match each dataset to its primary domain.
- Explain what county Federal Information Processing Standards (FIPS) codes are and why all datasets use them for merging.
- Describe the three open scholarship requirements: cite, reproduce, and share.

1. Social Determinants of Health (SDOH) as Research Variables

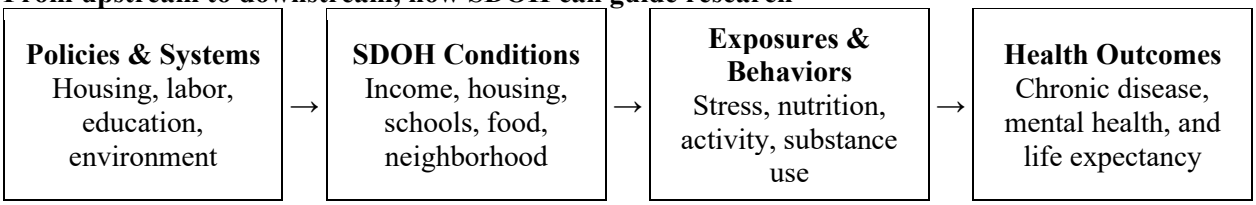
Health is not just about genes, personal choices, or access to doctors. Research consistently shows that where people are born, grow up, live, work, and age shapes their health outcomes more than medical care alone. These conditions are called social determinants of health, or SDOH.

> *"Social determinants of health are the conditions in the environments where people are born, live, learn, work, play, worship, and age that affect a wide range of health, functioning, and quality-of-life outcomes and risks."*
>
> *Healthy People 2030, U.S. Department of Health and Human Services*

SDOH are useful for open scholarship because they connect real-world social conditions to measurable indicators that can often be examined using publicly available data. Depending on the research question, indicators such as income, housing, education, food access, or neighborhood conditions may be used as independent variables, control variables, or outcomes of interest.

A useful way to think about this is that a person's health is influenced far upstream, long before they ever see a doctor. A child growing up in a neighborhood with high poverty, poor schools, food deserts, and environmental pollution faces a fundamentally different health trajectory than one who does not regardless of individual behavior.

From upstream to downstream, how SDOH can guide research



This toolkit focuses on exploring the SDOH conditions using publicly available data. By understanding where SDOH burdens are concentrated, researchers and students can identify communities most in need of policy intervention and support evidence-based decision-making.

2. The Five Healthy People 2030 SDOH Domains

Healthy People 2030 organizes SDOH into five domains. Every dataset in this toolkit maps to one or more of these domains. Understanding the domains helps you choose the right data for your research question.

SDOH Domain 1: Economic Stability

- Focuses on whether people have sufficient economic resources to maintain health, including income, employment, food security, housing stability, and financial strain.
- Example variables include poverty rate, unemployment rate, median household income, SNAP receipt, and housing cost burden.

SDOH Domain 2: Education Access & Quality

- Focuses on educational opportunity, school quality, early childhood education, literacy, and educational attainment.
- Example variables include high school graduation rate, educational attainment, school enrollment, chronic absenteeism, and limited English proficiency.

Domain 3: Health Care Access & Quality

- Focuses on insurance coverage, primary care access, provider shortages, preventive services, quality of care, and barriers to care, especially in rural and resource-limited areas.
- Example variables include uninsured rate, primary care shortage areas, cancer screening rates, preventable hospitalizations, and mental health providers.

SDOH Domain 4: Neighborhood & Built Environment

- Focuses on housing quality, food access, environmental pollution, transportation, walkability, parks, climate risk, and physical conditions of where people live.
- Example variables include food desert status, environmental burden index, housing cost burden, air quality/PM2.5, and disaster risk.

SDOH Domain 5: Social & Community Context

- Focuses on social support networks, civic participation, incarceration, community safety, social cohesion, and family and community structure.
- Example variables include violent crime rate, social associations, and voter participation

Note: These five domains do not operate separately. A community with high poverty often also has poor schools, fewer doctors, more pollution, and higher social vulnerability. These conditions tend to cluster geographically, which is why multi-domain analysis, combining data across all five domains, may be more powerful than studying any single domain alone. The datasets in this toolkit are designed to be merged and analyzed together.

3. The 10 Dataset Entries (8 Distinct Sources)

This toolkit utilizes 10 dataset entries from 8 sources. The American Community Survey (ACS) covers both Economic Stability and Education Access & Quality Domains. All sources are publicly available. Access methods vary by source, and some API-based workflows require a free API key. Geographic identifiers such as FIPS codes are standardized as needed before merging data in R.

What is the FIPS (Federal Information Processing Standards) code?

- A FIPS code is a 5-digit number that uniquely identifies U.S. counties. The first 2 digits represent the state, and the last 3 digits the county. For example, the FIPS code for Sangamon County, Illinois, is 17167. Here, 17 is Illinois, and 167 is Sangamon County.
- All datasets in this toolkit use FIPS codes to merge data in R.
- Store FIPS as a string, not a number, to keep leading zeros like Alabama's 01. Storing as a number removes zeros, causing data merge issues.

Domain 1 — Economic Stability Datasets

Dataset	What it measures	Geography	Format	Download URL
American Community Survey (ACS 5-Year) U.S. Census Bureau	Poverty, income, unemployment, SNAP receipt, housing cost burden, education, health insurance, disability, and language proficiency	Census Tract, Zip code tabulation area (**ZCTA**), County, State, National	CSV (Comma-Separated Values) files, API, FTP	Census Table: Income, Poverty & Employment
County Health Rankings & Roadmaps (CHR&R) Univ. of Wisconsin Population Health Institute	County-level health factors and outcomes, including unemployment, income inequality, child poverty, food insecurity, social associations, and violent crime	County, State, National	CSV, Excel, GitHub	National data & documentation

Domain 2 — Education Access & Quality Datasets

Dataset	What it measures	Geography	Format	Download URL
ACS 5-Year — Education variables U.S. Census Bureau	Educational attainment, school enrollment, preschool enrollment, limited English proficiency, and related household/social indicators	Census Tract, ZCTA, County, State, National	CSV, API, FTP	Census Table: Education
NCES Common Core of Data (CCD) National Center for Education Statistics	Public school and district characteristics, enrollment, free/reduced-price lunch eligibility, teacher/staff counts, school type, charter status, and Title I status	School, District, State, National	CSV	CCD Data Files

Domain 3 — Health Care Access & Quality Datasets

Dataset	What it measures	Geography	Format	Download URL

CDC PLACES: Local Data for Better Health Centers for Disease Control and Prevention	Measurements include local estimates of health outcomes, behaviors, status, prevention practices, and non-medical factors from the ACS.	Census Tract, ZCTA, County Place	CSV, API	PLACES Data Download PLACES Data Portal
HRSA Health Professional Shortage Areas Health Resources & Services Administration	Primary care, dental, and mental health shortage designations; HPSA score, status, type, and rural status	Service Address, County, State, National	CSV	HRSA Data Warehouse

Domain 4 — Neighborhood & Built Environment Datasets

Dataset	What it measures	Geography	Format	Download URL
ATSDR Environmental Justice Index (EJI) Agency for Toxic Substances and Disease Registry	Air quality, water quality, proximity to hazardous sites, climate burden, social vulnerability, health vulnerability — combined into composite EJI score	Census Tract, County	CSV, Geodatabase	EJI Data Download
USDA Food Access Research Atlas U.S. Department of Agriculture	Low-income and low-access food tracts, distance to nearest supermarket, share of households without vehicle access to food, food desert classification	Census Tract, County	CSV, Excel	Food Access Research Atlas – Download the Data

Domain 5 — Social & Community Context Datasets

Dataset	What it measures	Geography	Format	Download URL
ATSDR Social Vulnerability Index (SVI) Agency for Toxic Substances and Disease Registry	Overall vulnerability including (1) socioeconomic status, (2) Household Characteristics, (3) Racial & Ethnic Minority Status, (4) Housing Type & Transportation	Census Tract, ZCTA, County	CSV, Geodatabase	SVI Data & Documentation Download
CHR&R — Civic, Community and Social Support Univ. of Wisconsin Population Health Institute	Civic and community indicators such as voter turnout, social associations, broadband access, local news access, public libraries, violent crime, and related social support measures	County, State, National	CSV, Excel	National data & documentation

4. How To Use This Toolkit

All projects using this toolkit follow a consistent five-step workflow. These steps are the same whether you're an undergraduate working on a course project or an MPH student developing a capstone. Part 2 explains Steps 2 and 3 in detail, while Part 3 focuses on Steps 4 and 5.

1

Choose an SDOH domain and write a research question

Choose one or two HP 2030 domains. Then, formulate a research question that includes a specific population, location, and outcome. For example: "*Which Illinois counties with high poverty levels also have low cancer screening rates?*"

2

Download the data (Part 2)

Use the download links in Section 3. Save each raw file to data/raw/ without modifying it. Record the source URL, data year, and access date, as you will need all three for your citation.

3

Run the R scripts (Part 2)

Scripts #1 through #5 handle data downloading and cleaning for one or two datasets each. Script #6 merges all outputs into one master county CSV. Script #7 in Part 2 runs the statistical analyses.

4

Build your Tableau dashboard (Part 3)

Open Part 3 for step-by-step instructions on building your Tableau Public dashboard in your browser. Upload your master CSV, follow the guided exercises to build your maps, and publish your dashboard to Tableau Public. No software download needed.

5

Deposit and cite (Part 3)

Share your work openly through GitHub. When appropriate, create a citable research record with Zenodo and consider an institutional or other appropriate open repository.

5. Open Scholarship Requirements

This toolkit is part of a grant-funded open scholarship project. Every analysis you produce using these datasets should follow three open science practices so your work can be reused, verified, and built upon by others. Three open scholarship requirements for every project:

1. Cite your data. Every dataset you use must be cited in your methods section. Share your work openly on GitHub and cite all datasets used in your methods section.

2. Make it reproducible. Reproducible means someone who has never seen your project could download your files and get the same results without asking you any questions. **Six elements** make this possible:

A. **Raw data files:** Save every downloaded file to data/raw/ exactly as downloaded and never modify it. Clean the data in R and save separately to data/clean/. Never edit the original.

B. **Download dates:** Public datasets update without notice. Record the exact date you downloaded each file, the URLs where you downloaded directly, and the data year.
C. **R script with detailed comments:** Use # to explain what each section does and why. For example, "# Filter to Illinois only, change 'IL' for other states."
D. **Script run order:** Scripts must run in sequence. Someone reproducing your work needs to know how to run Script #1 before Script #5, or it will fail.
E. **Codebook:** List every variable in your master CSV: column name, plain-language label, source dataset, data year, and units. One row is for one variable.
F. **README file:** one document that ties it all together. Include your name, analysis description, script run order, R version, package versions, and download dates.

3. Share your work openly. The files you created in "**2. Make it reproducible**" should go into your GitHub repository. When your project is complete, create a free GitHub account at github.com and upload your project folder as a public repository. When appropriate for your project, you may create a citable research record through Zenodo and explore institutional or other open repository options. Step-by-step instructions for all three are in Part 3.

- Your repository should contain the final master CSV, R scripts, data dictionary, README file, and other shareable files needed for reproducibility. Keep original raw files locally unless redistribution is appropriate.
- If you are using this toolkit as part of a course, follow your instructor's directions for submitting your GitHub repository URL. If you are working independently, keep the public repository URL as part of your project record and portfolio. Selected student projects may also be featured in a course or public showcase.

The FAIR Principles

The three open scholarship requirements above (Cite, Reproduce, Share) are grounded in a widely used international framework called the **FAIR principles**, first published by Wilkinson et al. (2016) in *Scientific Data*. **FAIR** stands for:

- **F — Findable.** Your data and code should be easy for others to find. This means uploading your project to a public GitHub repository with a README and registering a permanent, citable identifier (DOI) through Zenodo, so others know what it contains.

- **A — Accessible.** Once found, your data and code should be retrievable by anyone without unnecessary barriers. Publishing your dashboard on Tableau Public, posting your scripts on GitHub as a public repository, and saving data in open file formats (CSV, not Excel) all make your work accessible.

- **I — Interoperable.** Your data should work with other datasets and tools. This is exactly why this toolkit uses FIPS codes to merge datasets. FIPS is a common, standardized identifier that links a geographic dataset to another. Using standard column names, consistent formats, and open file types (CSV, R scripts) all support interoperability.

- **R — Reusable.** Your data and code should be documented well enough that others, including yourself, can understand and reuse them. A well-commented R script, a complete codebook, dataset citations with download dates, and an open license all make your work reusable.

This toolkit is based on applying the FAIR principles. Using it helps you practice FAIR open scholarship.

FAIR	How Your Toolkit Already Teaches It	Part

Findable	Public project documentation, data dictionary, and a persistent identifier or repository record when appropriate.	Part 2 (README and data dictionary), Part 3 (GitHub, Zenodo DOI, IDEALS)
Accessible	Tableau Public dashboard, public GitHub repository, CSV open format	Part 3
Interoperable	FIPS codes for merging, standard column names, open file formats (CSV, R scripts)	Part 1 (FIPS concept) Part 2 (FIPS in all scripts)
Reusable	Commented R scripts, codebook/data dictionary, dataset citations with download dates	Part 2

Reference: Wilkinson, M. D., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, 3, 160018. https://doi.org/10.1038/sdata.2016.18

Dataset citation templates following APA 7th edition

Copy and paste these citations into the Data Sources section of your methods. Update the access date (in brackets) to the date you downloaded the data.

> **American Community Survey 5-Year**
> U.S. Census Bureau. (2026). American Community Survey 5-year estimates, 2020–2024 [Data set]. https://data.census.gov/

> **CDC PLACES 2025**
> Centers for Disease Control and Prevention. (2025). PLACES: Local data for better health, county data 2025 release [Data set]. https://data.cdc.gov/resource/swc5-untb

> **CDC/ATSDR Social Vulnerability Index 2022**
> Agency for Toxic Substances and Disease Registry. (2025). CDC/ATSDR Social Vulnerability Index 2022 [Data set]. https://www.atsdr.cdc.gov/place-health/php/svi/index.html

> **County Health Rankings & Roadmaps 2025**
> University of Wisconsin Population Health Institute. (2026). County Health Rankings & Roadmaps 2025 national data [Data set]. https://www.countyhealthrankings.org/health-data/methodology-and-sources/data-documentation

> **CDC/ATSDR Environmental Justice Index 2024**
> Centers for Disease Control and Prevention and Agency for Toxic Substances Disease Registry. (2024). Environmental Justice Index [Data set]. https://www.atsdr.cdc.gov/place-health/php/eji/index.html

> **USDA Food Access Research Atlas**
> U.S. Department of Agriculture, Economic Research Service. (2021). Food Access Research Atlas: 2019 data [Data set]. https://www.ers.usda.gov/data-products/food-access-research-atlas/

> **HRSA Health Professional Shortage Areas**

> Health Resources and Services Administration. (2024). Health Professional Shortage Areas (HPSAs) [Data set]. https://data.hrsa.gov/data/download

> **NCES Common Core of Data**
> National Center for Education Statistics. (2024). Common Core of Data [Data set]. https://nces.ed.gov/ccd/ccddata.asp

6. Part 1 Checklist

Complete all items below before moving on to Part 2. Items 1–4 are required before you start the R scripts in Part 2. Items 5–7 are best completed after you have downloaded your selected datasets.

- ☐ I can define SDOH in my own words using Healthy People 2030.
- ☐ I can name all five HP 2030 SDOH domains and give at least one example variable from each.
- ☐ I understand what FIPS codes are and why compatible geographic identifiers are important when merging datasets.
- ☐ I have reviewed the 10 dataset entries and have identified the datasets relevant to my research question.
- ☐ I have bookmarked or written down the download links for my selected datasets.
- ☐ I understand the three open scholarship requirements: cite, reproduce, and share.
- ☐ I have reviewed the five workflow steps and know what Part 2 and Part 3 cover.

If you cannot check items 1–4, return to the relevant section before starting Part 2. If you are stuck on a concept, re-read that section or consult your instructor, mentor, or the referenced learning resources. Do not move on until items 1–4 are complete.

Ready for the next step?

Part 2 covers preparing the toolkit datasets and running seven R scripts to produce a master analysis CSV and statistical analyses.

Part 2: Download & R Workflow →

Part 2. Download & R Workflow

API access with tidycensus and RSocrata, step-by-step R scripts for downloading, cleaning, and combining the toolkit datasets, followed by descriptive statistics, correlation, multiple regression, and logistic regression.

Prerequisite: Part 1 complete | **Level:** Beginner–Intermediate | **Time:** ~3–4 hours

Learning Objectives

- Set up a reproducible R project with the correct folder structure and all required packages.
- Download data using the Census API via `tidycensus`, `RSocrata`, and direct download.
- Merge all datasets into one clean master county-level CSV using `dplyr` joins.
- Run descriptive statistics, correlation matrix, and regression analyses.
- Export results, a data dictionary, and a README file for Tableau and open sharing on GitHub.

What you need before starting: install R and RStudio (free download) on your computer

- R language (base): https://cran.r-project.org/, download the version for your operating system (e.g., macOS, Windows).
- RStudio Desktop: https://posit.co/download/rstudio-desktop/, a free coding environment where you write and run all R scripts for this Part 2.
- Minimum recommended: R 4.3.0 or higher. Check your version with: `R.version$major`

1. R Environment Setup

Before running any scripts, complete the one-time setup steps below. This step needs to be completed only once per computer.

Project Folder Structure

Note: For the desktop version, begin by creating a folder on your desktop (or your desired Drive, such as C:\, D:\, or E:\ on your hard drive or USB drive) called **sdoh_toolkit**.

In RStudio, look up the RStudio Menu Bar at the top of the R Console. Then, select **File** > **New Project** > **Existing Directory.** Then, point your mouse to the desired drive containing the "**sdoh_toolkit**" folder, click the "**Browse…**" tab, and select the folder by pointing to it.

Then, click the "**Create Project**" tab.

Run the following R codes to build the Project Folder Structure without any files among all the folders:

```
# Create the entire folder structure from R (run once):
dirs <- c(
  "data/raw", "data/clean", "data/master",
  "code", "output/tableau", "output/figures", "output/tables",
  "dictionary"
)
sapply(dirs, dir.create, recursive = TRUE, showWarnings = FALSE)
cat("All folders created.\n")
```

Note. After you have completed all practices, the project folders may contain multiple files with the following Project Folder Structure with several layers of different types of files, such as data files, R codes/R scripts, R outputs, and data dictionary files as follows:

```
sdoh_toolkit/
  data/
    raw/          <- original downloaded files — NEVER modify these
    clean/        <- cleaned individual dataset files
    master/       <- final merged dataset (Script #6 output)
  code/           <- Paste each script into an R script and save it as an .R file
    01_acs.R
    02_places.R
    03_svi_eji.R
    04_hrsa.R
    05_usda_nces_chr.R
    06_merge_all.R
    07_analysis.R
  output/
    tableau/      <- final CSV for Tableau dashboard
    figures/      <- correlation plots, regression output
    tables/       <- descriptive stats, model results
  dictionary/
    data_dictionary.csv
```

Install Required R Packages

Copy and run this block once in your RStudio console. It installs every package used across all scripts.

```
# ── Step 1: Install all required packages (run once) ──────────────────────
install.packages(c(
  "tidycensus",    # ACS and Census API access
  "tidyverse",     # data wrangling (dplyr, ggplot2, readr, tidyr)
  "haven",         # read SAS/Stata files (.xpt, .dta)
  "readxl",        # read Excel files (.xlsx, .xls)
  "janitor",       # clean column names (clean_names())
  "skimr",         # enhanced descriptive statistics
  "corrplot",      # correlation matrix visualization
  "car",           # regression diagnostics (vif, Anova)
  "broom",         # tidy regression output (tidy(), glance())
  "lmtest",        # Breusch-Pagan test for homoscedasticity
  "ResourceSelection", # Hosmer-Lemeshow test for logistic regression
  "devtools", # required to install RSocrata from GitHub
  "sandwich", # robust standard errors
))

# ── Step 2: Install RSocrata from GitHub (run once) ──────────────────────
# RSocrata is maintained by the City of Chicago (github.com/Chicago/RSocrata)
# It is not available on CRAN for all R versions, so install from GitHub
# During installation, you may see two prompts:
# Question 1: "Would you like to install remotes?" -- select 1 (Yes)
# Question 2: "Which would you like to update?" -- select 1 (All)
#           If option 1 causes errors, re-run and select 3 (None)
devtools::install_github("Chicago/RSocrata")

# ── Step 3: Verify all packages loaded successfully ──────────────────────
# Verify all packages loaded successfully
library(tidycensus); library(tidyverse); library(RSocrata)
library(haven); library(readxl); library(janitor)
library(skimr); library(corrplot); library(car); library(broom)
library(lmtest); library(ResourceSelection)
```

Get Your Free Census API Key
The US Census API key is required for `tidycensus`, and the process takes about 2 minutes.

```
# Step 1: Request your free key at https://api.census.gov/data/key_signup.html
#         by typing your institution's name and your email address
# Step 2: Check your email. Your key arrives within a few minutes.
#         Remember to activate your API Key!
# Step 3: Install the API key in R (saves to your .Renviron; done once per machine)
library(tidycensus)
census_api_key("PUT_YOUR_KEY_HERE", install = TRUE, overwrite = TRUE)

# Step 4: Restart R first — Session menu > Restart R
# OR run this line instead of restarting:
readRenviron("~/.Renviron")

# Then verify your key is stored:
sys.getenv("CENSUS_API_KEY") # should print your key.
```

2. Data Download Scripts (Scripts #1–#5)

Scripts #1 through #5 download or prepare, clean, and save the source datasets used in the toolkit. All scripts follow the same structure: (1) load packages, (2) download via an API or direct URL, (3) clean and standardize FIPS codes, and (4) save the cleaned CSV to data/clean/. Run the scripts in order.

Script #1: ACS 5-Year for Domains 1 & 2
The American Community Survey (ACS) is the main dataset. This script uses the Census API with the `tidycensus` package to download variables for poverty, income, education, housing, insurance, and SNAP. Illinois at the county level is used as the default example throughout. Change the `state` and `geography` arguments to use any state or geographic unit.

Before running this script, you need to know the following things:

- **Where to find variable codes:** ACS has thousands of variables. Each has a code like `B17001_002`. To browse all available variables, run this in RStudio. Search by keyword (e.g., "poverty", "income", "education") to find the code for any variable you need. The Census Bureau also provides a full variable list at: api.census.gov/data/2024/acs/acs5/variables.html

```
vars <- load variables(2024, "acs5")
View(vars)   # opens a searchable table in RStudio
```

- **Where to get training:** The `tidycensus` package has free official documentation and tutorials at walker-data.com/tidycensus, written by the package author Kyle Walker. The U.S. Census Bureau also provides free video tutorials on using the Census API at census.gov/data/academy.

```
# ── Script #1: ACS 5-Year Download ──────────────────────────────────────
# Source:    U.S. Census Bureau, American Community Survey 5-year estimates
# API:       api.census.gov via tidycensus package
# Geography: County level, Illinois (state = 'IL')
# Year:      2024 (most recent 5-year ACS, 2020–2024)
# Citation:  U.S. Census Bureau. (2026). ACS 5-year estimates 2020-2024.
#            https://data.census.gov/
# ────────────────────────────────────────────────────────────────────────
```

```
library(tidycensus)
library(tidyverse)
library(janitor)

# ── Define variables to pull ──────────────────────────────────────────
# Variable codes: browse with load variables(2024, 'acs5') in RStudio
acs vars <- c(
  # Domain 1 — Economic Stability
  total pop          = "B17001 001",  # total population for poverty denom
  poverty count      = "B17001 002",  # persons below poverty line
  median_hh_income   = "B19013_001",  # median household income ($)
  unemployed         = "B23025 005",  # unemployed persons in labor force
  labor force        = "B23025 002",  # total labor force
  snap_hh            = "B22010_002",  # households receiving SNAP
  total hh           = "B22010 001",  # total households (SNAP denom)
  rent_burden_30plus = "B25070_007",  # gross rent 30-34.9% of income
  rent burden 35plus = "B25070 008",  # gross rent 35-39.9% of income
  rent burden 40plus = "B25070 009",  # gross rent 40-49.9% of income
  rent_burden_50plus = "B25070_010",  # gross rent 50%+ of income
  rent denom         = "B25070 001",  # renter-occupied units (denom)

  # Domain 2 — Education Access & Quality
  edu total 25plus   = "B15003 001",  # total pop 25+ (education denom)
  no hs diploma      = "B15003 002",  # less than 9th grade
  hs_graduate        = "B15003_017",  # HS graduate or equivalent
  bachelors          = "B15003 022",  # bachelor's degree
  graduate           = "B15003 023",  # master's degree
  professional       = "B15003_024",  # professional school degree
  doctorate          = "B15003 025",  # doctorate degree

  # Domain 3 — cross-domain variable (health insurance)
  uninsured pct = "DP03 0099P"  # % of residents without health insurance
                                # excludes prisoners and nursing home residents
)

# ── Download from Census API ──────────────────────────────────────────
acs_raw <- get_acs(
  geography = "county",
  variables = acs vars,
  state     = "IL",          # Change to 'all' for national data
  year      = 2024,
  survey    = "acs5",
  output    = "wide"          # one row per county, all vars as columns
)

# ── Clean and compute derived rates ──────────────────────────────────
acs clean <- acs raw %>%
  clean names() %>%
  rename(county_name = name) %>%
  mutate(
    # Standardize FIPS: always 5-digit character string with leading zeros
    fips = str_pad(geoid, width = 5, side = "left", pad = "0"),

    # Compute rates from raw counts
    poverty_rate     = round(poverty_count_e / total_pop_e * 100, 2),
    unemployment rate= round(unemployed e   / labor force e * 100, 2),
    snap_rate        = round(snap_hh_e       / total_hh_e   * 100, 2),
```

```
    # Rent cost burden: share of renters paying 30%+ of income on rent
    rent burden rate = round(
      (rent burden 30plus e + rent burden 35plus e +
       rent_burden_40plus_e + rent_burden_50plus_e) / rent_denom_e * 100, 2),

    # Education rates (% of adults 25+)
    pct_no_hs = round(no_hs_diploma_e / edu_total_25plus_e * 100, 2),
    pct hs grad = round(hs graduate e   / edu total 25plus e * 100, 2),
    pct bachelor plus = round(
      (bachelors_e + graduate_e + professional_e + doctorate_e) /
       edu total 25plus e * 100, 2),

    # Uninsured rate
    uninsured rate = uninsured pct e   # already a percentage from DP03 0099P
  ) %>%
  select(fips, county name, poverty rate, median hh income e,
         unemployment rate, snap rate, rent burden rate,
         pct_no_hs, pct_hs_grad, pct_bachelor_plus, uninsured_rate) %>%
  rename(median hh income = median hh income e)

# ── Inspect the output ──────────────────────────────────────────────────
glimpse(acs clean)
summary(acs clean)

# ── Save to data/clean/ ─────────────────────────────────────────────────
write csv(acs clean, "data/clean/01 acs clean.csv")
cat("Script #1 complete. Rows:", nrow(acs_clean), "\n")
```

You can use different geographies to conduct your analyses based on your research questions.

- **To change the geographic unit:** Replace geography = "county" with `geography = "tract"` for census tract data, or `geography = "zcta"` for ZIP code data.
- **To switch to national data:** Change `state = 'IL'` to `state = NULL` or remove the state argument entirely to pull all U.S. counties. The number of rows depends on your chosen geography: approximately 3,100 for county, 85,000 for census tract, and 33,000 for ZCTA.

Script #2: CDC PLACES 2025 for Domain 3

CDC PLACES provides model-based estimates for 40 health outcomes and behaviors. This script downloads the data directly into R using base R `read.csv()` from the CDC public API endpoint. No additional package or app token is required. Again, Illinois at the county level is used as the default example throughout.

```
# ─── Script #2: CDC PLACES 2025 County Data ──────────────────────────────
# Source:    Centers for Disease Control and Prevention
# Endpoint:  https://data.cdc.gov/resource/swc5-untb.csv
# Geography: County level — Illinois default. See end of script for tract/ZCTA
# Citation:  CDC. (2025). PLACES: Local data for better health, county data
#            2025 release. https://data.cdc.gov/resource/swc5-untb
# ─────────────────────────────────────────────────────────────────────────

library(tidyverse)
library(janitor)
```

```
# ─── Download PLACES data — Illinois counties only ──────────────────────
places_raw <- read.csv(
  "https://data.cdc.gov/resource/swc5-untb.csv?StateAbbr=IL&$limit=10000",
  stringsAsFactors = FALSE
)

# ─── Check the download ─────────────────────────────────────────────────
cat("Total rows downloaded:", nrow(places_raw), "\n")
names(places raw)
unique(places raw$measureid) %>% sort()

# ─── Check county coverage per measure ──────────────────────────────────
──────────────────────────

places_raw %>%
  filter(datavaluetypeid == "CrdPrv") %>%
  group_by(measureid) %>%
  summarise(n counties = n()) %>%
  arrange(n counties) %>%
  View()

# ─── Optional: Browse available measures ────────────────────────────────
# Run these lines to explore what measures are available before choosing
# Browse all measures with descriptions via searchable table in RStudio
places raw %>%
  select(measureid, measure, category) %>%
  distinct() %>%
  arrange(category, measureid) %>%
  View()

# Search by keyword — change "diabetes" to any topic you are interested in
places_raw %>%
  select(measureid, measure, category) %>%
  distinct() %>%
  filter(str_detect(tolower(measure), "diabetes"))

# ─── Select key measures and pivot to wide format ───────────────────────
measures_keep <- c(
  "DIABETES",
  "OBESITY",
  "BPHIGH",
  "DEPRESSION",
  "COPD",
  "CASTHMA",
  "CSMOKING",
  "LPA",
  "COLON_SCREEN",
  "MAMMOUSE",
  "ACCESS2"
)

places wide <- places raw %>%
  filter(measureid %in% measures_keep) %>%
  filter(datavaluetypeid == "CrdPrv") %>%
  select(locationid, locationname, measureid, data value) %>%
  mutate(
    fips       = str pad(locationid, width = 5, side = "left", pad = "0"),
    data_value = as.numeric(data_value)
```

```
  ) %>%
  pivot_wider(
    id_cols     = c(fips, locationname),
    names_from  = measureid,
    values_from = data_value
  ) %>%
  clean_names() %>%
  rename(
    diabetes_pct          = diabetes,
    obesity_pct           = obesity,
    hypertension_pct      = bphigh,
    depression_pct        = depression,
    copd_pct              = copd,
    asthma_pct            = casthma,
    smoking_pct           = csmoking,
    inactivity_pct        = lpa,
    colorectal_screen_pct = colon_screen,
    mammography_pct       = mammouse,
    uninsured_places      = access2
  )

cat("Counties in PLACES data:", nrow(places_wide), "\n")
glimpse(places_wide)
write_csv(places_wide, "data/clean/02_places_clean.csv")
cat("Script #2 complete. Rows:", nrow(places_wide), "\n")
```

Here's the information you need to download PLACES data at the census tract or ZCTA level, depending on your research question.

Census Tract Level
The tract file is already in wide format with one row per tract and all measures as separate columns named like `diabetes_crudeprev`. You do not need `pivot_wider()`. Use `tractfips` as the geographic identifier.

```
# ── Census tract and ZCTA options ──────────────────────────────────────
# Census tract — wide format, one row per tract, no pivot_wider() needed
# Use tractfips as the geographic identifier
places_tract <- read.csv(
  "https://data.cdc.gov/resource/yjkw-uj5s.csv?StateAbbr=IL&$limit=300000",
  stringsAsFactors = FALSE
)
names(places_tract)
```

ZCTA (ZIP Code Tabulation Area)
The ZCTA file is also in wide format. Use `zcta5` as the geographic identifier. ZCTAs do not align perfectly with county or tract boundaries; merging ZCTA data with other datasets requires a ZCTA-to-county crosswalk.

```
install.packages("CDCPLACES")
library(CDCPLACES)

il_zcta <- get_places(
  geography = "zcta",
  state     = "IL",
  measure   = c("DIABETES", "OBESITY", "BPHIGH",
                "DEPRESSION", "COPD", "CASTHMA",
```

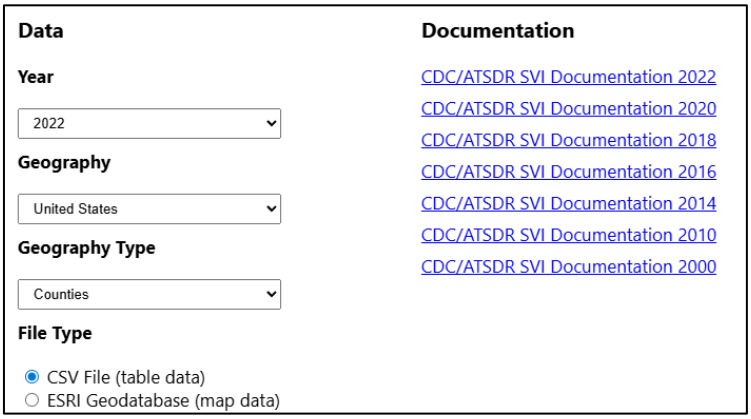
```
                "CSMOKING", "LPA", "COLON_SCREEN",
                "MAMMOUSE", "ACCESS2")
)

cat("Illinois ZCTA rows:", nrow(il_zcta), "\n")

# View structure and first few rows
glimpse(il_zcta)

# View column names
names(il_zcta)

# View first 10 rows in console
head(il_zcta, 10)

# Open searchable table in RStudio
View(il_zcta)

# Check available measures
unique(il_zcta$measure) %>% sort()
```

See full documentation at: `brendensm.github.io/CDCPLACES`

Script #3: CDC/ATSDR SVI 2022 & EJI 2024 for Domains 4 & 5

This script reads the SVI and EJI directly from downloaded CSV files from the ATSDR website. Download both files into data/raw/ before running. SVI offers county-level social vulnerability across 4 themes. The EJI provides tract-level environmental, social, and health vulnerability. The script aggregates tract-level EJI percentile values to the county level using unweighted means.

SVI 2022 County CSV: Save as: data/raw/SVI2022_US_county.csv



EJI 2024 CSV and file type should be .csv file. After the download is complete, extract the ZIP file and locate **EJI_2024_United_States.csv.** Save and rename this file as: data/raw/EJI_2024_US.csv

Note: If using the tract-level SVI file, the state column is named `stabbr` not `st_abbr`. Change the filter accordingly: `filter(stabbr == "IL")`.

```
# ── Script #3: SVI 2022 & EJI 2024 ──────────────────────────────────────
# SVI Source:  ATSDR. (2023). SVI 2022.
#              https://www.atsdr.cdc.gov/place-health/php/svi/index.html
# EJI Source:  CDC/ATSDR. (2024). Environmental Justice Index (EJI).
#              https://www.atsdr.cdc.gov/place-health/php/eji/index.html
# ──────────────────────────────────────────────────────────────────────────

library(tidyverse)
library(janitor)

# == PART A: Social Vulnerability Index (SVI) ==============================
```

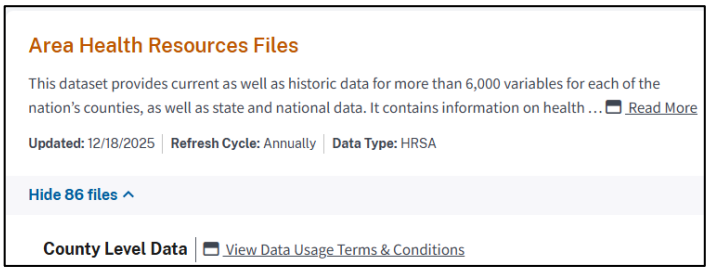
```
svi_raw <- read_csv("data/raw/SVI2022_US_county.csv")

svi_clean <- svi_raw %>%
  clean_names() %>%
  filter(st_abbr == "IL") %>%     # Change 'IL' to filter a different state
  mutate(
    # Pad FIPS to 5 digits — critical for merging
    fips = str_pad(as.character(fips), width = 5, side = "left", pad = "0"),

    # SVI flag: -999 means no data — replace with NA
    across(starts_with("rpl_"), ~ ifelse(. == -999, NA, .))
  ) %>%
  select(
    fips,
    county_name  = county,
    svi_overall  = rpl_themes,   # Overall SVI percentile (0-1; higher=more
vulnerable)
    svi_socioeco = rpl_theme1,   # Theme 1: Socioeconomic status
    svi_hh_char  = rpl_theme2,   # Theme 2: Household characteristics
    svi_minority = rpl_theme3,   # Theme 3: Racial/ethnic minority & language
    svi_housing  = rpl_theme4,   # Theme 4: Housing type & transportation
    # Key component variables
    pct_poverty  = ep_pov150,    # % persons below 150% poverty line
    pct_unemp_svi= ep_unemp,     # % civilian unemployed
    pct_uninsured_svi = ep_uninsur  # % uninsured
  )

cat("SVI rows (IL counties):", nrow(svi_clean), "\n")
write_csv(svi_clean, "data/clean/03a_svi_clean.csv")

# == PART B: Environmental Justice Index (EJI) ================================
# EJI is tract-level — aggregate to county by averaging

eji_raw <- read_csv("data/raw/EJI_2024_US.csv")

eji_county <- eji_raw %>%
  clean_names() %>%
  mutate(
    fips       = str_sub(geoid, 1, 5),
    state_abbr = stateabbr
  ) %>%
  filter(state_abbr == "IL") %>%
  mutate(across(c(rpl_eji, rpl_ebm, rpl_svm, rpl_hvm),
                ~ ifelse(. == -999, NA, .))) %>%
  group_by(fips) %>%
  summarise(
    eji_overall    = round(mean(rpl_eji, na.rm=TRUE), 4),  # Overall EJI
    eji_env_burden = round(mean(rpl_ebm, na.rm=TRUE), 4),  # Environmental Burden
    eji_social_vuln= round(mean(rpl_svm, na.rm=TRUE), 4),  # Social Vulnerability
    eji_health_vuln= round(mean(rpl_hvm, na.rm=TRUE), 4),  # Health Vulnerability
    n_tracts       = n()
  ) %>%
  ungroup()

cat("EJI county rows (IL):", nrow(eji_county), "\n")
write_csv(eji_county, "data/clean/03b_eji_county_clean.csv")
```

```
cat("Script #3 complete.\n")
```

Script #4: HRSA HPSA Shortage Areas for Domain 3
This script reads Health Professional Shortage Area (HPSA) designation data from the Area Health Resources Files (AHRF), published by HRSA. The AHRF is a county-level file that provides shortage designation status for primary care, dental, and mental health across all U.S. counties. Each county receives a designation code for each shortage type: 0 means no shortage designation, 1 means part of the county is designated, and 2 means the whole county is designated.

Before running this script, download the AHRF file:

1. Go to data.hrsa.gov/data/download
2. Click **Area Health Resources Files**
3. Under County, download **AHRF 2024-2025 County CSV**
4. Unzip the file and save AHRF2025.csv to data/raw/AHRF2025.csv



Note: AHRF is county-level only and cannot be broken down to census tract or ZIP code. If changed to tract-level geography in other scripts, omit Script #4 and remove the HPSA join from Script #6.

```
# ── Script #4: HRSA HPSA Shortage Areas from AHRF ──────────────────────────
# Source:  Health Resources & Services Administration
#          Area Health Resources Files (AHRF) 2024-2025
# URL:     https://data.hrsa.gov/data/download
# Steps:   1. Click "Area Health Resources Files"
#          2. Under County, download: AHRF 2024-2025 County CSV
#          3. Unzip and save AHRF2025.csv as: data/raw/AHRF2025.csv
# Citation: HRSA, Bureau of Health Workforce. (2025). Area Health Resources
#           Files 2024-2025. https://data.hrsa.gov/data/download
# Note:    AHRF is county-level only. Cannot be disaggregated to tract or ZIP.
#          HPSA variables use designation codes:
#          0 = No shortage designation
#          1 = Part of county is designated (partial shortage)
#          2 = Whole county is designated (full shortage)
# ──────────────────────────────────────────────────────────────────────────

library(tidyverse)
library(janitor)

# ── Load AHRF county file ──────────────────────────────────────────────────
ahrf_raw <- read_csv("data/raw/AHRF2025.csv")

# ── Inspect column names ───────────────────────────────────────────────────
names(ahrf_raw) %>% sort()

# ── Filter to Illinois and select HPSA variables ──────────────────────────
hpsa_clean <- ahrf_raw %>%
  clean_names() %>%
  filter(st_name_abbrev == "IL") %>%   # Change "IL" for other states
  mutate(
    fips = str_pad(as.character(fips_st_cnty), 5, "left", "0")
```

```
  ) %>%
  select(
    fips,
    county name         = cnty name,
    # 2025 HPSA designation codes (0=none, 1=partial, 2=full)
    hpsa prim care      = hpsa prim care 25,   # Primary care shortage
    hpsa dental         = hpsa dent 25,         # Dental shortage
    hpsa_mental_health  = hpsa_mentl_hlth_25    # Mental health shortage
  ) %>%
  mutate(
    # Convert to numeric
    across(c(hpsa prim care, hpsa dental, hpsa mental health), as.numeric),
    # Create binary shortage indicator (1 = any shortage, 0 = none)
    hpsa_any_shortage   = as.integer(
      hpsa prim care > 0 | hpsa dental > 0 | hpsa mental health > 0),
    # Create primary care binary (used in logistic regression)
    hpsa prim care designated = as.integer(hpsa prim care > 0)
  )

# ── Check distribution ──────────────────────────────────────────────────
cat("HPSA rows (IL counties):", nrow(hpsa clean), "\n")

# Primary care shortage distribution
cat("\nPrimary care HPSA designation (0=none, 1=partial, 2=full):\n")
table(hpsa_clean$hpsa_prim_care)

cat("\nDental HPSA designation:\n")
table(hpsa_clean$hpsa_dental)

cat("\nMental health HPSA designation:\n")
table(hpsa_clean$hpsa_mental_health)

cat("\nCounties with ANY shortage designation:",
    sum(hpsa_clean$hpsa_any_shortage), "\n")

# ── Save ────────────────────────────────────────────────────────────────
write_csv(hpsa_clean, "data/clean/04_hpsa_clean.csv")
cat("Script #4 complete.\n")
```

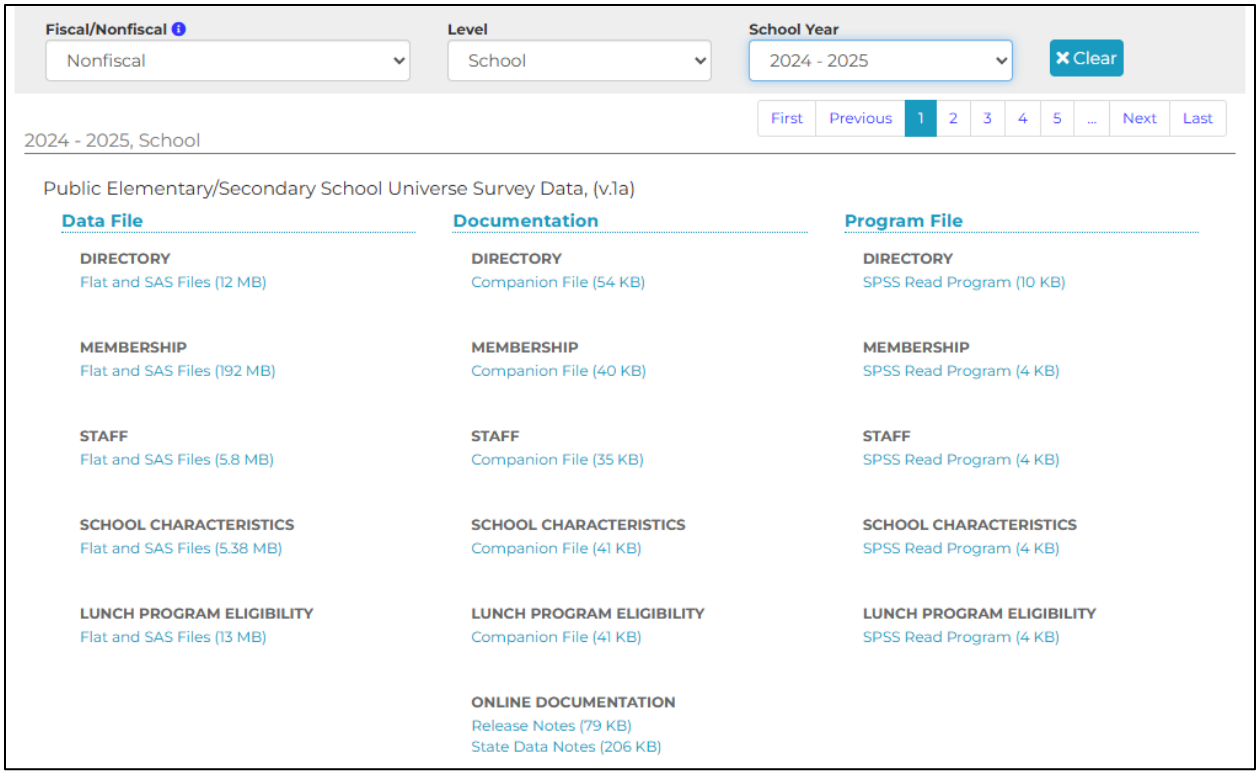
Script #5: USDA Food Access, NCES CCD & County Health Rankings for Domains 1, 2 & 5
This script reads three datasets covering food access, education, and community health.

The USDA Food Access Research Atlas provides food desert classifications at the census tract level, aggregated to the county level. The most current USDA Food Access data year is 2019. Download the Excel file from Food Access Research Atlas and save as data/raw/FoodAccessResearchAtlasData.xlsx

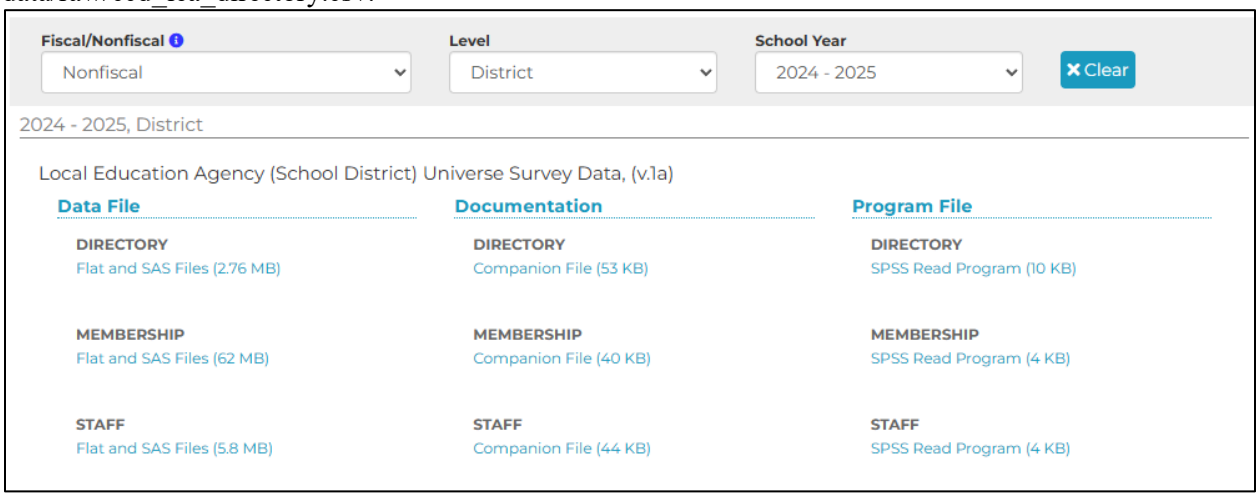
County Health Rankings & Roadmaps (CHR&R) provides pre-calculated county-level health outcomes and civic engagement variables. Download the 2025 national data CSV from [County Health Rankings Data & Documentation] and save as data/raw/2025CHR_CSV_Analytic_Data.csv.

The NCES Common Core of Data (CCD) provides school enrollment data by district, aggregated to the county level. Two files are required. Go to [nces.ed.gov/ccd/files.asp] and download:

(1) Nonfiscal > School > 2024-2025 > LUNCH PROGRAM ELIGIBILITY (13 MB): save CSV as data/raw/ccd_lunch.csv



(2) Nonfiscal > District > 2024-2025 > DIRECTORY (2.76 MB): save CSV as data/raw/ccd_lea_directory.csv.



Note. Illinois does not report free and reduced-price lunch counts, but 46 other states do. This script automatically detects whether your state reports this data.

```
# ——— Script #5: USDA Food Access + NCES CCD + County Health Rankings ——————
# Download before running:
# USDA: ers.usda.gov/data-products/food-access-research-atlas/download-the-data
#       Save as: data/raw/FoodAccessResearchAtlasData.xlsx
#       Note: Most current data year is 2019.
# CHR&R: countyhealthrankings.org/health-data/methodology-and-sources/data-
documentation
```

```
#        Save as: data/raw/2025CHR CSV Analytic Data.csv
# NCES CCD: nces.ed.gov/ccd/files.asp(District file, most recent year)
#   Download 1: Nonfiscal > School > 2024-2025 > LUNCH PROGRAM ELIGIBILITY
#               Unzip and save CSV as: data/raw/ccd lunch.csv
#   Download 2: Nonfiscal > District > 2024-2025 > DIRECTORY
#               Unzip and save CSV as: data/raw/ccd lea directory.csv
#   Note: Free/reduced lunch counts not available for all states.
#         Illinois does not report these. 46 other states do.
#   Note: ZIP-to-county crosswalk may assign some districts to multiple
#         counties. Enrollment counts are used as descriptive context only.
# ──────────────────────────────────────────────────────────────────────────

library(tidyverse)
library(readxl)
library(janitor)

# ═══ PART A: USDA Food Access Research Atlas ═══════════════════════════
usda raw <- read excel(
  "data/raw/FoodAccessResearchAtlasData.xlsx",
  sheet = "Food Access Research Atlas"
)

# ─── Check column names before cleaning ──────────────────────────────────
usda raw %>% clean names() %>% names() %>%
  .[grep("lila|lapop|lahunv|tract|state", .)] %>%
  head(20)


# ── Then run the cleaning code ──────────────────────────────────────────
usda county <- usda raw %>%
  clean_names() %>%
  filter(state == "Illinois") %>%
  mutate(
    fips = str_pad(as.character(census_tract), 11, "left", "0") %>%
           str sub(1, 5),
    lapophalfshare = as.numeric(lapophalfshare),
    lahunv1share   = as.numeric(lahunv1share)
  ) %>%
  group by(fips) %>%
  summarise(
    pct lila tracts       = round(mean(lila tracts 1and10, na.rm=TRUE) * 100, 2),
    mean dist supermarket = round(mean(lapophalfshare,      na.rm=TRUE), 4),
    pct_no_vehicle_far    = round(mean(lahunv1share,        na.rm=TRUE) * 100, 2)
  ) %>%
ungroup()

cat("USDA county rows:", nrow(usda county), "\n")
write csv(usda county, "data/clean/05a usda food access clean.csv")

# ═══ PART B: County Health Rankings 2025 ═══════════════════════════════
chr raw <- read csv(
  "data/raw/2025CHR_CSV_Analytic_Data.csv",
  skip = 1
)
chr_clean <- chr_raw %>%
  clean names() %>%
  filter(state == "IL",
```

```
        fipscode != "17000") %>%       # remove state summary row
  mutate(
    fips = str pad(as.character(fipscode), 5, "left", "0")
  ) %>%
  select(
    fips,
    premature death     = v001 rawvalue,
    poor_health_days    = v036_rawvalue,
    poor mental days    = v042 rawvalue,
    social associations = v140 rawvalue,
    voter_turnout       = v153_rawvalue,
    adult smoking       = v009 rawvalue,
    adult obesity       = v011 rawvalue,
    food_insecurity     = v139_rawvalue
  ) %>%
  mutate(
    across(everything(), as.numeric),
    # Convert proportions to percentages
    voter_turnout   = voter_turnout   * 100,
    adult smoking   = adult smoking   * 100,
    adult obesity   = adult obesity   * 100,
    food_insecurity = food_insecurity * 100
  )

# Note: Column codes change with each annual CHR&R release.
# Run names(chr raw) and check the CHR codebook to verify codes.
cat("CHR&R county rows:", nrow(chr clean), "\n")
write_csv(chr_clean, "data/clean/05b_chr_clean.csv")

# ═══ PART C: NCES CCD Lunch Program Eligibility ═══════════════════════

# ─── Set your state FIPS here ──────────────────────────────────────────
STATE FIPS <- "17"   # Illinois = 17. Change for other states.

# ─── Download Census TIGER ZIP-to-county crosswalk ─────────────────────
tiger url <- "https://www2.census.gov/geo/docs/maps-
data/data/rel2020/zcta520/tab20_zcta520_county20_natl.txt"

zip county xwalk <- read.delim(tiger url, sep = "|") %>%
  clean_names() %>%
  select(geoid zcta5 20, geoid county 20) %>%
  rename(zip = geoid zcta5 20, fips = geoid county 20) %>%
  mutate(
    zip  = as.character(zip),
    fips = as.character(fips)
  ) %>%
  filter(str sub(fips, 1, 2) == STATE FIPS) %>%
  distinct()

cat("ZIP-county pairs downloaded:", nrow(zip county xwalk), "\n")

# ─── Load district directory — use ZIP to get county FIPS ──────────────
dir raw <- read.csv("data/raw/ccd lea directory.csv",
                    stringsAsFactors = FALSE) %>%
  clean_names() %>%
  filter(as.character(fipst) == STATE FIPS) %>%
  mutate(
```

```
    leaid = as.character(leaid),
    zip   = str_pad(as.character(mzip), 5, "left", "0")
  )

# Valid IL county FIPS — Illinois uses odd numbers only (17001 to 17203)
valid il fips <- sprintf("17%03d", seq(1, 203, by = 2))

# Join ZIP crosswalk — allow many-to-many for full county coverage
ccd dir <- dir raw %>%
  left join(zip county xwalk, by = "zip",
            relationship = "many-to-many") %>%
  filter(!is.na(fips), fips %in% valid il fips) %>%
  select(leaid, fips) %>%
  distinct()

cat("Districts with valid county FIPS:", nrow(ccd_dir), "\n")
cat("Unique counties:", n distinct(ccd dir$fips), "\n")

# ── Load lunch file ──────────────────────────────────────────────────────
ccd lunch <- read.csv("data/raw/ccd lunch.csv",
                      stringsAsFactors = FALSE) %>%
  clean_names() %>%
  filter(as.character(fipst) == STATE FIPS)

cat("School lunch rows:", nrow(ccd_lunch), "\n")

# ── Check if your state reports free lunch counts ────────────────────────
lunch_check <- ccd_lunch %>%
  filter(lunch program == "Free lunch qualified",
         total indicator == "Category Set A",
         !is.na(student_count)) %>%
  nrow()

cat("Schools with free lunch counts:", lunch_check, "\n")

if (lunch check == 0) {
  cat("NOTE: Your state does not report free lunch counts in this file.\n")
  cat("Only total enrollment and district count will be available.\n")
  cat("Food insecurity is captured via CHR&R food insecurity variable.\n")
} else {
  cat("Free lunch data available. Full aggregation will proceed.\n")
}

# ── Extract total enrollment ─────────────────────────────────────────────
total enroll <- ccd lunch %>%
  filter(data_group == "Free and Reduced-price Lunch Table",
         lunch program == "No Category Codes",
         total indicator == "Education Unit Total") %>%
  select(ncessch, leaid, student_count) %>%
  rename(total students = student count) %>%
  mutate(
    total_students = as.numeric(total_students),
    leaid          = as.character(leaid)
  )

# ── Extract free lunch counts ────────────────────────────────────────────
free_lunch <- ccd_lunch %>%
```

```
  filter(data group == "Free and Reduced-price Lunch Table",
         lunch_program == "Free lunch qualified",
         total indicator == "Category Set A") %>%
  select(ncessch, student count) %>%
  rename(free_lunch_count = student_count) %>%
  mutate(free lunch count = as.numeric(free lunch count))

# ── Extract reduced lunch counts ──────────────────────────────────────
reduced lunch <- ccd lunch %>%
  filter(data group == "Free and Reduced-price Lunch Table",
         lunch_program == "Reduced-price lunch qualified",
         total indicator == "Category Set A") %>%
  select(ncessch, student count) %>%
  rename(reduced_lunch_count = student_count) %>%
  mutate(reduced lunch count = as.numeric(reduced lunch count))

# ── Join at school level ──────────────────────────────────────────────
school lunch <- total enroll %>%
  left_join(free_lunch,    by = "ncessch") %>%
  left join(reduced lunch, by = "ncessch")

# ── Join district directory to get county FIPS ────────────────────────
school county <- school lunch %>%
  mutate(leaid = as.character(leaid)) %>%
  left_join(ccd_dir, by = "leaid",
            relationship = "many-to-many")

cat("Schools with county FIPS:", sum(!is.na(school_county$fips)), "\n")
cat("Unique counties in schools:", n distinct(school county$fips, na.rm=TRUE),
"\n")

# ── Load valid county FIPS from ACS for filtering ─────────────────────
acs fips <- read csv("data/clean/01 acs clean.csv",
                     col_types = cols(fips = col_character())) %>%
  pull(fips)

# ── Aggregate to county level ─────────────────────────────────────────
ccd county <- school county %>%
  filter(!is.na(fips), fips %in% acs fips) %>%
  group_by(fips) %>%
  summarise(
    n districts      = n distinct(leaid),
    total_enrollment = sum(total_students,      na.rm = TRUE),
    free lunch       = sum(free lunch count,     na.rm = TRUE),
    reduced lunch    = sum(reduced lunch count,  na.rm = TRUE)
  ) %>%
  ungroup() %>%
  mutate(
    pct_free_lunch    = ifelse(lunch_check > 0,
                               round(free lunch / total enrollment * 100, 2),
                               NA real ),
    pct_reduced_lunch = ifelse(lunch_check > 0,
                               round(reduced lunch / total enrollment * 100, 2),
                               NA real )
  ) %>%
  select(fips, n districts, total enrollment, pct free lunch, pct reduced lunch)
```

```
cat("NCES CCD county rows:", nrow(ccd county), "\n")
glimpse(ccd_county)
write csv(ccd county, "data/clean/05c nces ccd clean.csv")
cat("Script #5 complete. Rows:", nrow(ccd_county), "\n")
```

3. Merge All Datasets into Master CSV (Script 6)

The following script merges the eight cleaned source files produced by Scripts #1 through #5 into a single master analysis CSV. This master file is the input for Tableau (Part 3) and for all statistical analyses in Section 4 below.

Note: CHR&R, HRSA, and NCES data are county-level only. If you change to tract or ZCTA geography in Scripts #1–#3, remove those joins from this script and exclude their variables from your analysis. Additionally, the NCES file uses a ZIP-to-county crosswalk, which may slightly inflate enrollment counts for counties with districts on county boundaries. This is expected and does not affect other variables.

```
# ── Script #6: Merge All Datasets — Master County CSV ──────────────────────
# Input:  data/clean/01 through 05 CSV files
# Output: data/master/sdoh_il_master.csv
# ──────────────────────────────────────────────────────────────────────────

library(tidyverse)

# ── Load all clean files ──────────────────────────────────────────────────
acs      <- read_csv("data/clean/01_acs_clean.csv")
places   <- read_csv("data/clean/02_places_clean.csv")
svi      <- read csv("data/clean/03a svi clean.csv")
eji      <- read_csv("data/clean/03b_eji_county_clean.csv")
hpsa     <- read_csv("data/clean/04_hpsa_clean.csv")
usda     <- read csv("data/clean/05a usda food access clean.csv")
chr      <- read_csv("data/clean/05b_chr_clean.csv")
nces <- read csv("data/clean/05c nces ccd clean.csv")

# ── Standardize all FIPS columns as character ─────────────────────────────
acs   <- acs   %>% mutate(fips = as.character(fips))
places <- places %>% mutate(fips = as.character(fips))
svi   <- svi   %>% mutate(fips = as.character(fips))
eji   <- eji   %>% mutate(fips = as.character(fips))
hpsa  <- hpsa  %>% mutate(fips = as.character(fips))
usda  <- usda  %>% mutate(fips = as.character(fips))
chr   <- chr   %>% mutate(fips = as.character(fips))
nces  <- nces  %>% mutate(fips = as.character(fips))

# ── Merge all on FIPS (left join — keep all ACS counties as base) ─────────
# Check row counts before and after each join to detect FIPS mismatches
cat("ACS base rows:", nrow(acs), "\n")

master <- acs %>%
  left_join(places %>% select(-locationname), by = "fips") %>%
  { cat("After PLACES:", nrow(.), "\n"); . } %>%
  left join(svi    %>% select(-county name),  by = "fips") %>%
  { cat("After SVI:",    nrow(.), "\n"); . } %>%
  left join(eji,                             by = "fips") %>%
  { cat("After EJI:",    nrow(.), "\n"); . } %>%
  left_join(hpsa   %>% select(-county_name),  by = "fips") %>%
```

```
  { cat("After HPSA:",   nrow(.), "\n"); . } %>%
  left_join(usda,                          by = "fips") %>%
  { cat("After USDA:",   nrow(.), "\n"); . } %>%
  left join(chr,                           by = "fips") %>%
  { cat("After CHR:",    nrow(.), "\n"); . } %>%
  left join(nces,                          by = "fips") %>%
  { cat("After NCES:",   nrow(.), "\n"); . }

# ── Verify: row count should equal number of IL counties (102) ──────────────
stopifnot(nrow(master) == nrow(acs))   # stops script if rows changed

# ── Quick data quality check ────────────────────────────────────────────────
cat("\nMissing values per column:\n")
colSums(is.na(master)) %>% sort(decreasing = TRUE) %>% head(15) %>% print()

# ─── Set output file label ──────────────────────────────────────────────────
# Change "il" to your state abbreviation if using a different state
STATE LABEL <- "il"   # e.g. "ca" for California, "tx" for Texas

# ─── Save master file ───────────────────────────────────────────────────────
master filename <- paste0("data/master/sdoh ", STATE LABEL, " master.csv")
tableau_filename <- paste0("output/tableau/sdoh_", STATE_LABEL, "_tableau.csv")

write csv(master, master filename)
write_csv(master, tableau_filename)

cat("\nScript #6 complete.\n")
cat("Master file saved:", master_filename, "\n")
cat("Tableau file saved:", tableau filename, "\n")
cat("Rows:", nrow(master), "| Columns:", ncol(master), "\n")
```

Diagnosing FIPS Merge Problems
If row counts change after a join, you have a FIPS mismatch. Common causes: (1) FIPS stored as numeric in one file (losing leading zeros), (2) FIPS is 4 digits in one file instead of 5, (3) a dataset has fewer counties than expected. Fix: always run `str_pad(as.character(fips), 5, 'left', '0')` on the FIPS column in every cleaning script before merging.

4. Statistical Analyses

This section explains the concepts to understand before running your statistical analysis, including the purpose of each method, appropriate usage scenarios, the required assumptions, and methods for verifying those assumptions.

Requirement Before Starting
Script #6 must have run successfully and produced your master CSV file in data/master/. The file name reflects your chosen state. For example, if your state is Illinois, your file is named sdoh_il_master.csv. All analyses in this section read from that file. Verify your file exists and has the correct row count by running:

```
STATE LABEL <- "il"   # change to match your state
master <- read csv(paste0("data/master/sdoh ", STATE LABEL, " master.csv"))
cat("Rows:", nrow(master), "| Columns:", ncol(master), "\n")
```

Statistical Foundations

This toolkit introduces the most commonly used statistical methods to analyze how SDOH relate to community outcomes. These methods can support research questions across different fields. For example, "Is poverty associated with higher diabetes rates?" or "Which SDOH factors best predict poor mental health outcomes?" Statistical method selection should be guided by your research question and the type of data you have.

What is Regression Analysis?
Regression analysis is a statistical method that models the relationship between an outcome variable (dependent variable) and one or more predictor variables (independent variables). In SDOH research, we generally use regression for two primary purposes: Explanation or Prediction.

Explanatory Questions: Use this approach when you want to understand how specific SDOH factors relate to a health outcome, while accounting for confounding variables. You can focus on a single primary exposure or a set of exposures. Examples include: "Is a higher poverty rate linked to increased diabetes prevalence, after adjusting for education level and insurance coverage?" and "Are counties with low civic engagement more likely to have poor mental health outcomes?" The question format is:

> *"Are [Independent Variable(s) of Interest] associated with [Dependent Variable], after controlling for [Confounders/Covariates]?"*

Predictive Questions: Use this approach when you want to identify which combination of multiple SDOH factors best forecasts or explains the variance in a population health outcome. Example includes: "Which combination of SDOH factors best explains and predicts variation in premature death rates across Illinois counties?" The question formula is:

> *"Which combination of [Candidate Predictor Variables] best explains and predicts variation in [Dependent Variable] across [Geographic/Unit Level]?"*

Keep in mind that although regression can predict outcomes, it does not establish causation. It identifies statistical associations. Two variables can be strongly correlated or predictive without one causing the other. Always interpret regression results in the context of existing theory and prior literature.

Outcome Variable vs. Predictor Variables
Before choosing a regression model, you must identify independent and dependent variables.

Outcome Variable (Y)	**Predictor Variables (X)**
Also called: dependent variable, response variable, Y variable	Also called: independent variables, explanatory variables, X variables
The **health outcome** you are trying to explain or predict.	The **SDOH factors** you believe are associated with the outcome.
Examples in this toolkit: • Diabetes prevalence (%) • Depression prevalence (%) • Premature death rate • Poor mental health days • Physical inactivity (%)	**Examples in this toolkit:** • Poverty rate (Domain 1) • % with bachelor's degree (Domain 2) • Uninsured rate (Domain 3) • EJI environmental burden (Domain 4) • Social Vulnerability Index (Domain 5)

Descriptive Statistics
Descriptive statistics offer a summary and overview of your dataset before modeling. They must be the initial step in any analysis. Standard regression models (e.g., linear, logistic) are sensitive to outliers, skewed distributions, and data entry errors. For example, if the maximum value for the poverty rate is 9,500%, you may have a data entry error. Always check the minimum, maximum, and standard deviation before fitting any model.

Statistical Metric	What Descriptive Statistics Tell Us
N (sample size)	How many counties have complete data for each variable? Missing data reduces your effective sample size.
Mean	The average value across all counties. For example, the average diabetes prevalence across all 102 Illinois counties.
Standard deviation (SD)	How spread out the values are. A high SD means counties vary widely; a low SD means they are similar to each other.
Min and Max	The most extreme values. Check for implausible values (e.g., a poverty rate of 200%) that indicate data errors.
Median	The middle value. If they differ greatly from the mean, the distribution is skewed.
Missing values	Variables with many missing values may produce unreliable model results. Investigate why the data are missing before proceeding.

Multiple Linear Regression: Concept and Interpretation

Multiple linear regression is used when your outcome variable is continuous, such as a percentage, rate, or score. It models how multiple predictor variables jointly relate to that outcome. The linear regression equation:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k + \varepsilon$$

Where:

- Y: The outcome variable (e.g., diabetes prevalence %)
- β_0 (intercept): The predicted value of Y when all predictors equal zero
- β_1, β_2, ... β_n (coefficients): The change in Y for a one-unit increase in each predictor, holding all other predictors constant.
- X_1, X_2, ... X_n: The predictor variables (e.g., poverty rate, education level)
- ε (error term): The residual, the part of Y not explained by the model

How to Interpret a Linear Regression Coefficient

Once you run your regression model, you must evaluate the outputs, including the direction and magnitude, statistical significance, and total model fit.

Output value	What it means	Plain-language example
Coefficient (β): Direction and Magnitude. Coefficients (β) examine variables individually. For each one-unit increase in predictor X, the outcome Y changes by β units, holding all other predictors constant. If the Coefficient (β) is...		
Positive (e.g., +0.18)	For each 1-unit increase in *X*, *Y* increases by 0.18 units. Units: X = %; Y = %	Poverty Rate (β = 0.18): Each 1 percentage point increase in county poverty rate is associated with a 0.18 percentage point higher diabetes prevalence.
Negative (e.g., -0.22)	For each 1-unit increase in *X*, *Y* decreases by 0.22 units Units: X = %; Y = %	Bachelor's Degree (β = -0.22): Each 1 percentage point increase in college education is associated with a 0.22 percentage point lower diabetes prevalence.
***p*-value and 95% Confidence Interval (CI): Statistical Significance.** Interpret the coefficient together with its confidence interval and p-value. A non-significant estimate can still be interpreted, but its uncertainty should be acknowledged.		
$p < 0.05$	Association is statistically significant at the 95% confidence level	$p = 0.003$: There is a 0.3% chance of seeing this result if the true relationship were zero.

$p > 0.05$	Association is not statistically significant	$p = 0.42$: Result is not significant; the predictor may not be associated with the outcome.
95% CI does not include 0	Range of plausible coefficient values does not cross zero. Indicates statistical significance.	CI [0.05, 0.31] is significant CI [−0.04, 0.40] is not significant (crosses zero)
R^2 (R-Squared): Total Model Fit. R^2 examines the model. It represents the percentage of variance in the outcome explained by predictors.		
R^2 (e.g., 0.65)	Percentage of variance in the outcome explained by all predictors combined	$R^2 = 0.65$: Predictors explain 65% of the county-to-county variation in diabetes prevalence; 35% is unexplained.

Controlling for Other Variables

The phrase "holding all other predictors constant" highlights the advantage of multiple regression over simple correlation. Let's think about what we know. Poverty rate and low education are strongly correlated with each other and with diabetes. A simple correlation cannot separate these effects. However, multiple regression estimates the independent contribution of each predictor after removing the shared influence of the others. This explains why a multiple regression coefficient can differ from a simple correlation.

Assumptions of Multiple Linear Regression

For linear regression results to be valid, six assumptions must be evaluated. Violating an assumption does not always mean discarding your results, but it does mean acknowledging the violation and adjusting your approach.

1 **Linearity**

- **What it means:** The relationship between each predictor and the outcome is a straight line.
- **Why it matters:** Linear regression fits a straight line through your data.
- **How to check:** Residuals vs Fitted plot. Points should scatter randomly around the horizontal zero line with no curve or pattern.
- **What to do if violated:** Log-transform the outcome variable (log(Y)) if it is right-skewed or add a squared term (X^2) to model the curve.

2 **Independence of observations**

- **What it means:** Each observation is independent; knowing the value for one geographic area does not tell you about the value for its neighbor.
- **Why it matters:** If geographic areas are spatially clustered (neighboring areas share similar health outcomes), this assumption is violated.
- **How to check:** For area-level data, this is largely guided by study design. Advanced researchers test spatial autocorrelation using Moran's I test (`sf` and `spdep` R packages).
- **What to do if violated:** Address this as a limitation in your methods section. For this toolkit's scope, standard regression is acceptable with this caveat.

3 **Homoscedasticity (equal variance of residuals)**

- **What it means:** The spread of residuals (prediction errors) is roughly the same across all predicted values.
- **Why it matters:** If residuals fan out (heteroscedasticity), the standard errors are incorrect, leading to wrong *p*-values and confidence intervals.
- **How to check:** Scale-Location plot (a flat red line indicates equal variance) and the Breusch-Pagan test ($p > 0.05$ indicates the assumption is met).
- **What to do if violated:** Use heteroscedasticity-robust standard errors via the `sandwich` R package: `coeftest(model, vcov = vcovHC(model))`. The coefficients remain identical; only the standard errors and *p*-value are adjusted.

4

Normality of residuals

- **What it means:** The differences between the model's predictions and the actual values (the residuals) follow a normal distribution.
- **Why it matters:** The *p*-values and CIs are calculated assuming normally distributed residuals. Linear regression is robust to mild non-normality, especially with a sample size of 100 or more, due to the central limit theorem.
- **How to check:** Normal Q-Q plot (points should fall along the diagonal line) and Shapiro-Wilk test ($p > 0.05$ indicates the assumption is met).
- **What to do if violated:** If the Q-Q plot shows heavy skew, try log-transforming the outcome variable: `lm(log(outcome) ~ predictors)`. Back-transform coefficients when reporting.

5

No multicollinearity

- **What it means:** Predictor variables are not too highly correlated with each other. It is expected and ok if they correlate with the outcome.
- **Why it matters:** When two predictors carry nearly the same information, the model cannot reliably estimate each predictor's contribution. Coefficients become unstable, and standard errors inflate dramatically. Including both poverty rate and SVI socioeconomic score causes the model to explain poverty twice, making estimates unreliable.
- **How to check:** Variance Inflation Factor (VIF). VIF < 5 is acceptable. VIF 5–10 is a moderate concern. VIF > 10 is a serious problem, and the predictor needs to be removed.
- **What to do if violated:** Remove the predictor with the highest VIF and re-run the model. Repeat until all VIF < 5.

6

No highly influential outliers

- **What it means:** No single geographic area is excessively driving the regression results on its own.
- **Why it matters:** An outlier geographic area with unusual traits, like high poverty but low diabetes rates, can significantly influence the regression line. Your conclusion should reflect overall geographic patterns, not be dictated by one or two anomalies.
- **How to check:** Cook's Distance. A common threshold is 4/n (where n is the number of geographic areas). Geographic areas above this threshold are flagged as potentially influential.

$$D_i = \frac{e_i^2}{p \cdot MSE} \times \frac{h_{ii}}{(1 - h_{ii})^2}$$

- **What to do if violated:** Run the model with and without the flagged areas. If the results change substantially, report both models and discuss the influential observation in your limitations section.

Logistic Regression: Concept and Interpretation
Logistic regression is used when your outcome variable is binary, such as yes/no, high/low, or above threshold/below threshold. You cannot use linear regression for a binary outcome because it can predict values outside the 0–1 range.

The logistic regression model
Instead of predicting the outcome value, logistic regression predicts the probability that the outcome equals 1. For example, the probability that a county has a high diabetes rate.

$$log(\frac{P}{1 - P}) = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k$$

The left side is the log-odds (also called the logit) of the outcome occurring. The right side is the same linear combination of predictors in linear regression. Coefficients are estimated in log-odds units, then converted to Odds Ratios (OR) for interpretation.

What is an Odds Ratio?
The Odds Ratio (OR) is the output of logistic regression. It measures the change in the odds of the outcome for a one-unit increase in the predictor, holding all other variables constant. It is calculated by exponentiating the raw model coefficient:

$$OR = e^{\beta}$$

Odds Ratio Interpretation Table
Outcome Variable (*Y*): Whether a county falls into the top 50% of diabetes prevalence in the state (1 = Yes, 0 = No).

Output Metric	What it Means	Plain-Language Example
OR = 1.0	No association	Voter turnout (OR = 1.0): No association with the odds of the county falling into the top half of diabetes prevalence.
OR > 1.0 (e.g., 1.25)	Higher predictor value is associated with higher odds of the outcome	Poverty rate (OR = 1.25): Each 1 percentage point increase in county poverty rate is associated with 25% higher odds of the county falling into the top half of diabetes prevalence.
OR < 1.0 (e.g., 0.78)	Higher predictor value is associated with lower odds of the outcome.	Bachelor's degree (OR = 0.78): Each 1 percentage point increase in college graduation is associated with 22% lower odds of the county falling into the top half of diabetes prevalence.
***p* < 0.05**	Statistically significant association	*p* = 0.01: There is only a 1% chance of observing this result if the true population association were zero.
95% CI excludes 1.0	Another way to confirm significance	CI [1.05, 1.52] does not include 1.0 → significant; CI [0.95, 1.38] includes 1.0 → not significant

How the Binary Outcome is Created in This Toolkit

For continuous outcome variables (percentages, rates, or scores), the script creates a binary outcome by comparing each geographic area to the state median. Variables that are already binary, such as `hpsa_any_shortage`, can be used as the outcome without this conversion step.

```
# Areas above the state median diabetes rate → high_diabetes = 1
# Areas at or below the state median        → high diabetes = 0
# Example for Illinois (median diabetes ≈ 11.5%):
# Cook County    diabetes = 10.2%  →  high_diabetes = 0 (below median)
# Alexander County diabetes = 17.8% →  high diabetes = 1 (above median)
```

Assumptions of Logistic Regression
Logistic regression has fewer distributional assumptions than linear regression, but it has its own requirements that must be checked.

1

Binary outcome

- **What it means:** The outcome must only have two values, 0 or 1. Logistic regression requires a binary outcome. Continuous outcomes should ordinarily be analyzed using an appropriate continuous-outcome model. In this toolkit, median dichotomization is used as a teaching demonstration when a naturally binary outcome is unavailable. For example, coding an outcome as 1 if it is above the median and 0 if it is below.
- **How to check:** Verify your outcome variable containing two unique values using R: `table(logistic_df$outcome_high)`

2

Independence of observations

- **What it means:** Each geographic unit must be an independent observation.
- **How to check:** Review the study design. Advanced analyses, such as spatial autocorrelation, may be necessary. If not, this should be stated in the limitations section or addressed through spatial analyses (outside this handout's scope).

3

No multicollinearity

- **What it means:** Predictor variables should not be too highly correlated with each other. For example, both poverty and an overall SVI might cause multicollinearity because SVI is partly derived from ACS poverty data.
- **How to check:** Variance Inflation Factor (VIF). Just like linear regression, a VIF < 5 is acceptable for all predictors.

4

No complete or quasi-complete separation

- **What it means:** Separation occurs when predictors perfectly split the binary outcome, predicting whether an observation is 0 or 1. This can cause unstable model estimates, extremely large odds ratios, and inflated standard errors. It is more likely with small samples, rare outcomes, or threshold-based outcomes.
- **How to check:** Look for very large coefficients that are greater than 10 on the log-odds scale in your R output.

5

Adequate sample size (events per variable)

- **What it means:** Logistic regression needs enough counties with outcome = 1 and outcome = 0 for each predictor. A common rule of thumb is at least 10 events per predictor variable. For example, with about 51 high-diabetes counties and 7 predictors, 51 / 7 ≈ 7 events per predictor, which is borderline. To improve model stability, use 5 or fewer predictors.
- **How to check:** Calculates EPV = `n_events / length(PREDICTOR_VARS).`

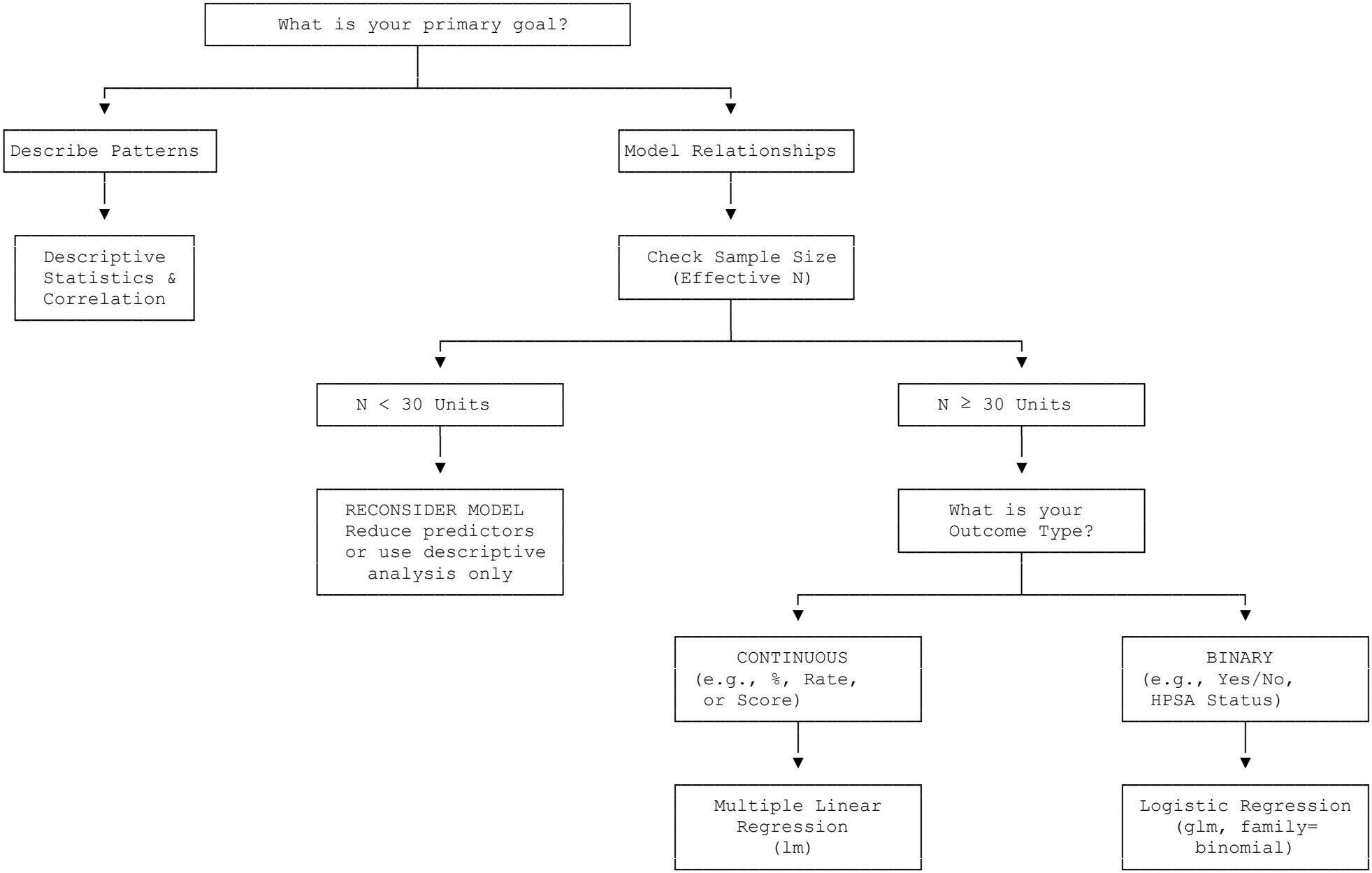
6

Good model fit

- **What it means:** The model's predicted probabilities should match the observed outcomes adequately. In other words, the model should be a good fit for the data. If not, predicted probabilities will be inaccurate and misleading.
- **How to check:** Hosmer-Lemeshow test. $p > 0.05$ indicates adequate fit. $p < 0.05$ means the model may need revision.

Quick guide: which regression should I use?

- My outcome is a percentage, rate, or score (e.g., diabetes %, premature death rate) → **Use multiple linear regression (lm)**
- My outcome is yes/no, above/below a threshold, or a binary designation (e.g., HPSA designated county) → **Use logistic regression (glm, family=binomial)**
- I want to describe patterns without making causal claims → **Use both: descriptive statistics and correlation matrix first, then regression**
- I have fewer than 30 counties after removing missing values → **Reconsider your model — reduce the number of predictors or consider descriptive analysis only**
- I want to compare one outcome across all five HP 2030 domains simultaneously → **Run separate models for each domain pair, or run one fully adjusted model with one predictor per domain**


What is your primary goal?
Describe Patterns
Model Relationships
Descriptive Statistics & Correlation
Check Sample Size (Effective N)
N < 30 Units
N ≥ 30 Units
RECONSIDER MODEL Reduce predictors or use descriptive analysis only
What is your Outcome Type?
CONTINUOUS (e.g., %, Rate, or Score)
BINARY (e.g., Yes/No, HPSA Status)
Multiple Linear Regression (lm)
Logistic Regression (glm, family= binomial)

Flexible Variable Selection
All analyses in Script #7 are read from a single configuration block at the top of the script. You only need to edit this one block to change your outcome variable, your predictors, or both. The rest of the script runs automatically with your chosen variables.

How To Choose Your Variables
- Outcome variable (*Y*) must be the variable you are trying to explain or predict.
  - For linear regression, it must be continuous (e.g., `diabetes_pct`, `depression_pct`).
  - For logistic regression, continuous outcomes will be converted to binary inside the script.
- Predictor variables (*X*): the SDOH factors you believe are associated with your outcome. Choose variables from different domains to test a multi-domain model. Avoid using two variables that measure nearly the same thing (e.g., `poverty_rate` AND `pct_poverty_svi` together).
  - For linear regression, predictors must be continuous.
  - For logistic regression, predictors can be continuous, categorical, or a mix of both.

Rule of thumb for sample size: you need at least 10-20 observations per predictor variable. For example, at the county level, Illinois has 102 observations, which comfortably supports approximately 5-7 predictor variables. Adjust this rule based on your chosen state and geographic unit. See the variable reference table for the full list of available variables and their domains.

```
# ═══════════════════════════════════════════════════════════════════════════
# Script #7: Statistical Analyses
# ═══════════════════════════════════════════════════════════════════════════
# EDIT THIS BLOCK ONLY — change your outcome and predictor variables here.
# Everything else in the script uses these names automatically.
# ═══════════════════════════════════════════════════════════════════════════

library(tidyverse)
library(skimr)
library(corrplot)
library(car)
library(broom)
library(lmtest)   # Breusch-Pagan test for homoscedasticity
library(ResourceSelection)  # Hosmer-Lemeshow test for logistic regression

# ── Load master dataset ──────────────────────────────────────────────────
STATE_LABEL <- "il"   # must match what you set in Script 06
master <- read_csv(paste0("data/master/sdoh_", STATE_LABEL, "_master.csv"))

# ═══════════════════════════════════════════════════════════════════════════
# VARIABLE CONFIGURATION — EDIT THIS SECTION TO CHANGE YOUR ANALYSIS
# ═══════════════════════════════════════════════════════════════════════════

# ── Step 1: Choose your OUTCOME variable (Y) ─────────────────────────────
# Must be a continuous variable for linear regression.
# It will be dichotomized automatically for logistic regression.
#
# Available outcome options (change the value in quotes below):
#   "diabetes_pct"        — Diagnosed diabetes prevalence (%)
#   "depression_pct"      — Depression prevalence (%)
#   "obesity_pct"         — Adult obesity prevalence (%)
#   "hypertension_pct"    — High blood pressure prevalence (%)
#   "smoking_pct"         — Current smoking prevalence (%)
#   "inactivity_pct"      — Physical inactivity prevalence (%)
```

```
#   "premature death"     — Premature death rate (YPLL per 100k)
#   "poor_mental_days"    — Avg mentally unhealthy days/month
#   "poor health days"    — Avg physically unhealthy days/month
#
OUTCOME_VAR <- "diabetes_pct"   # <── CHANGE THIS

# ── Step 2: Choose your PREDICTOR variables (X) ──────────────────────────
# List 3-7 variables. More than 7 is too many for n=102 counties.
#
# Available predictor options:
#  -- Domain 1: Economic Stability --
#   "poverty rate"        — % persons below poverty line
#   "unemployment rate"   — % civilian labor force unemployed
#   "median_hh_income"    — median household income ($)
#   "snap rate"           — % households receiving SNAP
#   "rent_burden_rate"    — % renters paying 30%+ income on rent
#
#  -- Domain 2: Education --
#   "pct_no_hs"           — % adults 25+ with less than HS diploma
#   "pct bachelor plus"   — % adults 25+ with bachelor's degree or higher
#   "pct free lunch"      — % students eligible for free lunch (NCES CCD)
#   "total_enrollment"    — total student enrollment in county (NCES CCD)
#
#  -- Domain 3: Health Care Access --
#   "uninsured_rate"            — % population uninsured (ACS)
#   "hpsa prim care"            — primary care shortage (0=none, 1=partial, 2=full)
#   "hpsa dental"               — dental shortage (0=none, 1=partial, 2=full)
#   "hpsa_mental_health"       — mental health shortage (0=none, 1=partial, 2=full)
#   "hpsa any shortage"         — any shortage designation (1=yes, 0=no)
#   "hpsa prim care designated" — primary care shortage binary (1=yes, 0=no)
#
#  -- Domain 4: Built Environment --
#   "pct lila tracts"     — % tracts that are low-income & low-access food
#   "eji_overall"         — EJI overall environmental burden percentile
#   "eji env burden"      — EJI environmental burden percentile
#
#  -- Domain 5: Social & Community Context --
#   "svi overall"         — SVI overall vulnerability percentile (0-1)
#   "svi socioeco"        — SVI Theme 1: socioeconomic status
#   "svi_minority"        — SVI Theme 3: minority status & language (as defined by
#                                        CDC/ATSDR SVI documentation)
#   "social associations" — membership organizations per 100k
#   "voter_turnout"       — % voting-age pop who voted
#
PREDICTOR VARS <- c(
  "poverty_rate",         # <── EDIT THIS LIST
  "pct bachelor plus",
  "uninsured rate",
  "pct_lila_tracts",
  "social associations"
)

# ── Step 3 (optional): Add a label for your output files ─────────────────
ANALYSIS LABEL <- "diabetes sdoh model"   # <── CHANGE TO DESCRIBE YOUR MODEL

# ── Build analysis dataset from your chosen variables ────────────────────
all_vars   <- c("fips", "county_name", OUTCOME_VAR, PREDICTOR_VARS)
```

```
analysis df <- master %>%
  select(all_of(all_vars)) %>%
  drop na()

cat("\n── Analysis configuration ──────────────────────────────\n")
cat("Outcome:    ", OUTCOME VAR, "\n")
cat("Predictors: ", paste(PREDICTOR VARS, collapse=", "), "\n")
cat("Sample size:", nrow(analysis_df), "observations (after removing missing)\n")
cat("Counties dropped due to missing:", nrow(master) - nrow(analysis df), "\n")
```

Variable Reference Table

Use this table to choose your outcome and predictor variables. All variable names listed here are exact column names in the master CSV file.

Variable name	Label	Domain	Type	Source
diabetes_pct	Diagnosed diabetes (%)	Health outcome	Continuous	CDC PLACES 2025
depression_pct	Depression (%)	Health outcome	Continuous	CDC PLACES 2025
obesity_pct	Adult obesity (%)	Health outcome	Continuous	CDC PLACES 2025
hypertension_pct	High blood pressure (%)	Health outcome	Continuous	CDC PLACES 2025
smoking_pct	Current smoking (%)	Health outcome	Continuous	CDC PLACES 2025
inactivity_pct	Physical inactivity (%)	Health outcome	Continuous	CDC PLACES 2025
premature_death	Premature death rate (YPLL/100k)	Health outcome	Continuous	CHR&R 2025
poor_mental_days	Mentally unhealthy days/mo	Health outcome	Continuous	CHR&R 2025
poor_health_days	Physically unhealthy days/mo	Health outcome	Continuous	CHR&R 2025
poverty_rate	% below poverty line	Domain 1	Continuous	ACS 2020-2024
unemployment_rate	% unemployed	Domain 1	Continuous	ACS 2020-2024
median_hh_income	Median household income ($)	Domain 1	Continuous	ACS 2020-2024
snap_rate	% HH receiving SNAP	Domain 1	Continuous	ACS 2020-2024
rent_burden_rate	% renters cost-burdened 30%+	Domain 1	Continuous	ACS 2020-2024
pct_no_hs	% adults <HS diploma	Domain 2	Continuous	ACS 2020-2024
pct_bachelor_plus	% adults bachelor's degree+	Domain 2	Continuous	ACS 2020-2024
n_districts	Number of school districts	Domain 2	Count	NCES CCD 2024
total_enrollment	Total student enrollment	Domain 2	Count	NCES CCD 2024
pct_free_lunch	% students free lunch eligible	Domain 2	Continuous	NCES CCD 2024
pct_reduced_lunch	% students reduced lunch eligible	Domain 2	Continuous	NCES CCD 2024

uninsured_rate	% population uninsured	Domain 3	Continuous	ACS 2020-2024
hpsa_prim_care	Primary care HPSA designation (0=none, 1=partial, 2=full)	Domain 3	Ordinal	AHRF 2025
hpsa_dental	Dental HPSA designation (0=none, 1=partial, 2=full)	Domain 3	Ordinal	AHRF 2025
hpsa_mental_health	Dental HPSA designation (0=none, 1=partial, 2=full)	Domain 3	Ordinal	AHRF 2025
hpsa_any_shortage	Dental HPSA designation (0=none, 1=partial, 2=full)	Domain 3	Binary	AHRF 2025
hpsa_prim_care_designated	Primary care shortage (1=yes, 0=no)	Domain 3	Binary	AHRF 2025
pct_lila_tracts	% low-income & low-access tracts	Domain 4	Continuous	USDA 2019
eji_overall	EJI overall percentile (0-1)	Domain 4	Continuous	EJI 2024
eji_env_burden	EJI environmental burden	Domain 4	Continuous	EJI 2024
svi_overall	SVI overall percentile (0-1)	Domain 5	Continuous	SVI 2022
svi_socioeco	SVI Theme 1: socioeconomic	Domain 5	Continuous	SVI 2022
svi_minority	SVI Theme 3: minority & language	Domain 5	Continuous	SVI 2022
social_associations	Social associations per 100k	Domain 5	Continuous	CHR&R 2025
voter_turnout	Voter turnout %	Domain 5	Continuous	CHR&R 2025

Avoid Collinear Predictors

- Do not include both `svi_overall` and `poverty_rate` in the same model. They are strongly correlated (SVI is partly derived from ACS poverty data). Similarly, avoid combining highly overlapping measures without first reviewing correlations and VIFs.
- Run the correlation matrix first. Treat an absolute correlation above about .70 as a warning. Review correlations, VIFs, substantive knowledge, and the research question before deciding whether to remove a predictor.

Descriptive Statistics Table

Always start with descriptive statistics. This gives you a complete picture of the distribution, missing values, and range of every variable before running any models.

```
# == PART C: Descriptive Statistics ============================================
# Uses OUTCOME VAR and PREDICTOR VARS defined in the configuration block above.

# Full skim of all key variables
skim_output <- analysis_df %>%
```

```
  select(all of(c(OUTCOME VAR, PREDICTOR VARS))) %>%
  skim()
print(skim output)

# ── Formatted summary table ──────────────────────────────────────────
desc stats <- analysis df %>%
  select(all of(c(OUTCOME VAR, PREDICTOR VARS))) %>%
  pivot_longer(everything(), names_to="variable", values_to="value") %>%
  group by(variable) %>%
  summarise(
    n       = sum(!is.na(value)),
    missing = sum(is.na(value)),
    mean    = round(mean(value,   na.rm=TRUE), 2),
    sd      = round(sd(value,     na.rm=TRUE), 2),
    min     = round(min(value,    na.rm=TRUE), 2),
    median  = round(median(value, na.rm=TRUE), 2),
    max     = round(max(value,    na.rm=TRUE), 2)
  )

print(desc stats)
write csv(desc stats,
  paste0("output/tables/01_descriptive_", ANALYSIS_LABEL, ".csv"))
```

Correlation Matrix

Run the correlation matrix before regression. It reveals which predictors are strongly associated with the outcome AND with each other. High correlations between predictors ($r > 0.70$) indicate multicollinearity and require dropping one of the correlated variables.

```
# ═══ PART D: Correlation Matrix ═══════════════════════════════════════

cor_data <- analysis_df %>%
  select(all_of(c(OUTCOME_VAR, PREDICTOR_VARS))) %>%
  drop_na()

cor_matrix <- cor(cor_data, method="pearson")

# ── Plot ─────────────────────────────────────────────────────────────
corrplot(
  cor_matrix,
  method      = "color",
  type        = "upper",
  order       = "hclust",
  addCoef.col = "black",
  tl.col      = "black",
  tl.srt      = 45,
  number.cex  = 0.75,
  title       = paste("Correlation Matrix —", OUTCOME_VAR),
  mar         = c(0,0,2,0)
)

# Save plot
png(paste0("output/figures/02_correlation_", ANALYSIS_LABEL, ".png"),
    width=1400, height=1200, res=150)
corrplot(cor_matrix, method="color", type="upper", order="hclust",
         addCoef.col="black", tl.col="black", tl.srt=45, number.cex=0.75)
dev.off()
```

```
# ── Flag high inter-predictor correlations ──────────────────────────────
pred_cor <- cor(analysis_df %>% select(all_of(PREDICTOR_VARS)), method="pearson")
high_pairs <- which(abs(pred_cor) > 0.70 & upper.tri(pred_cor), arr.ind=TRUE)

if (nrow(high_pairs) > 0) {
  cat("\nWARNING — High correlations (r > 0.70) between predictors:\n")
  for (i in seq_len(nrow(high_pairs))) {
    r1 <- rownames(pred_cor)[high_pairs[i,1]]
    r2 <- colnames(pred_cor)[high_pairs[i,2]]
    r  <- round(pred_cor[high_pairs[i,1], high_pairs[i,2]], 2)
    cat(" ", r1, "vs", r2, ":", r, "\n")
  }
  cat("ACTION: Consider removing one variable from each highly correlated pair.\n")
} else {
  cat("\nNo high inter-predictor correlations (all r < 0.70). Proceed.\n")
}
```

Why We Run Both Unadjusted and Adjusted Models

This toolkit runs both unadjusted and adjusted models for linear and logistic regression. For logistic regression, presenting unadjusted (crude) and adjusted odds ratios side by side is a well-established standard in epidemiological reporting. Bagley et al. (2001) in the Journal of Clinical Epidemiology recommend that both unadjusted and adjusted estimates always be presented so readers can assess the degree of confounding. For linear regression, although reporting practices vary across the health research literature, presenting unadjusted coefficients alongside adjusted coefficients serves an important pedagogical and analytical purpose. It shows how much each predictor's association with the outcome changes after controlling for other variables, which captures the concept of confounding in SDOH research. For each predictor, an unadjusted model is run separately with only that predictor and the outcome. Then, a fully adjusted model is run with all predictors together. Comparing the two reveals which predictors are independently associated with the outcome, and which associations are explained by other SDOH factors.

References

- Bagley, S. C., White, H., & Golomb, B. A. (2001). Logistic regression in the medical literature: Standards for use and reporting, with particular attention to one medical domain. *Journal of Clinical Epidemiology*, 54(10), 979-985. https://doi.org/10.1016/S0895-4356(01)00372-9
- Jones, L., Barnett, A., & Vagenas, D. (2025). Linear regression reporting practices for health researchers, a cross-sectional meta-research study. *PLOS ONE*. https://doi.org/10.1371/journal.pone.0305150
- Sperandei, S. (2014). Understanding logistic regression analysis. *Biochemia Medica*, 24(1), 12-18. https://doi.org/10.11613/BM.2014.003

Multiple Linear Regression with Full Assumption Checks

Multiple linear regression requires five assumptions to hold for the results to be valid. This section fits the model and systematically checks each assumption using both statistical tests and diagnostic plots.

Five Assumptions of Multiple Linear Regression

1. **Linearity —** Each predictor has a linear relationship with the outcome. Check: residuals vs fitted plot. Points should scatter randomly around zero with no curve.
2. **Independence —** Observations are independent of each other. Check: confirmed by study design. Each observation is a separate unit.
3. **Homoscedasticity —** Residual variance is constant across fitted values. Check: Scale-Location plot & Breusch-Pagan test. Violation: use robust standard errors.

4. **Normality of residuals —** Residuals follow a normal distribution. Check: Q-Q plot & Shapiro-Wilk test.
5. **No influential outliers —** No single county is excessively driving the results. Check: Cook's Distance plot. Observations with Cook's D > 4/n are flagged for review.

```
# ═══ PART E: Multiple Linear Regression + Assumption Checks ═══════════════

# ─── Unadjusted models — one per predictor ──────────────────────────────
# Standard epidemiological practice: run one unadjusted model per predictor
# Each gives a crude regression coefficient (unadjusted β)
cat("\n─── Unadjusted linear models (crude β — one predictor each) ───\n")

unadj_lm_results <- map_dfr(PREDICTOR_VARS, function(var) {
  formula_unadj <- as.formula(paste(OUTCOME_VAR, "~", var))
  model_unadj   <- lm(formula_unadj, data = analysis_df)
  tidy(model_unadj, conf.int = TRUE) %>%
    filter(term == var) %>%
    mutate(model = "Unadjusted", predictor = var,
           across(where(is.numeric), ~round(., 4)))
})

print(unadj_lm_results %>%
        select(predictor, estimate, conf.low, conf.high, p.value))

# ─── Fully adjusted model — all predictors together ─────────────────────
cat("\n─── Fully adjusted model (all predictors) ───\n")
formula_adj <- as.formula(paste(OUTCOME_VAR, "~",
                                paste(PREDICTOR_VARS, collapse=" + ")))
model_adj   <- lm(formula_adj, data = analysis_df)
summary(model_adj)

# ═══════════════════════════════════════════════════════════════════════════
# ASSUMPTION CHECKS — MULTIPLE LINEAR REGRESSION
# ═══════════════════════════════════════════════════════════════════════════

cat("\n══ ASSUMPTION CHECKS: MULTIPLE LINEAR REGRESSION ══\n")

# ── Assumption 1 & 3 & 5: Diagnostic plots ──────────────────────────────
# Four plots: (1) Residuals vs Fitted, (2) Normal Q-Q,
#             (3) Scale-Location,       (4) Residuals vs Leverage
png(paste0("output/figures/03_lm_diagnostics_", ANALYSIS_LABEL, ".png"),
    width=1600, height=1400, res=150)
par(mfrow=c(2,2))
plot(model_adj, main=paste("LM Diagnostics —", OUTCOME_VAR))
par(mfrow=c(1,1))
dev.off()
cat("Diagnostic plots saved to output/figures/\n")
cat("Interpret: (1) Residuals vs Fitted — no pattern = linearity + homoscedasticity
OK\n")
cat("           (2) Q-Q plot — points on line = normality OK\n")
cat("           (3) Scale-Location — flat red line = homoscedasticity OK\n")
cat("           (4) Cook's Distance — no points > dashed line = no influential
outliers\n")

# ── Assumption 3: Breusch-Pagan test for homoscedasticity ────────────────
cat("\n── Assumption 3: Homoscedasticity (Breusch-Pagan test) ──\n")
```

```
bp test <- bptest(model adj)
print(bp_test)
if (bp test$p.value < 0.05) {
  cat("RESULT: p <0.05 — heteroscedasticity detected (unequal variance).\n")
  cat("ACTION: Use heteroscedasticity-consistent (robust) standard errors.\n")
  cat("        Run: library(sandwich); coeftest(model adj,
vcov=vcovHC(model adj))\n")
} else {
  cat("RESULT: p >", round(bp test$p.value,3), "— homoscedasticity assumption
met.\n")
}

# ── Assumption 4: Shapiro-Wilk test for normality of residuals ──────────────
cat("\n── Assumption 4: Normality of residuals (Shapiro-Wilk test) ──\n")
sw test <- shapiro.test(residuals(model adj))
print(sw_test)
if (sw test$p.value < 0.05) {
  cat("RESULT: p <0.05 — residuals may not be normally distributed.\n")
  cat("NOTE:   With n=102, linear regression is generally robust to mild\n")
  cat("        non-normality. Check the Q-Q plot visually.\n")
  cat("        If Q-Q plot shows heavy skew, consider log-transforming the
outcome.\n")
} else {
  cat("RESULT: p >", round(sw test$p.value,3), "— normality assumption met.\n")
}

# ── Assumption 5: Cook's Distance — influential observations ────────────────
cat("\n── Assumption 5: Influential observations (Cook's Distance) ──\n")
cooks d    <- cooks.distance(model adj)
cutoff     <- 4 / nrow(analysis df)   # common threshold: 4/n
influential <- which(cooks_d > cutoff)
cat("Cook's D threshold (4/n):", round(cutoff, 4), "\n")
if (length(influential) > 0) {
  cat("Influential observations (Cook's D >", round(cutoff,4), "):\n")
  print(analysis df[influential, c("fips","county name", OUTCOME VAR)])
  cat("ACTION: Run sensitivity analyses - see Follow-up 2 below.\n")
} else {
  cat("No influential outliers detected.\n")
}

# ── Multicollinearity: Variance Inflation Factor (VIF) ──────────────────────
cat("\n── Multicollinearity check: Variance Inflation Factor (VIF) ──\n")
cat("VIF < 5 = acceptable | VIF 5-10 = moderate concern | VIF > 10 = serious
problem\n")
vif vals <- vif(model adj)
print(round(vif_vals, 2))
if (any(vif vals > 5)) {
  cat("WARNING: VIF > 5 detected. Consider removing correlated predictors.\n")
  cat("High VIF variables:", names(which(vif_vals > 5)), "\n")
} else {
  cat("VIF OK — no multicollinearity problem detected.\n")
}

# ── Model results table ─────────────────────────────────────────────────────
cat("\n── Final regression results ──\n")
results lm <- tidy(model adj, conf.int=TRUE) %>%
  mutate(across(where(is.numeric), ~round(.,4)))
```

```
print(results lm)

fit lm <- glance(model adj)
cat("R² =", round(fit lm$r.squared,3),
    " | Adj. R² =", round(fit_lm$adj.r.squared,3),
    " | F-stat p =", round(fit lm$p.value,4), "\n")

write_csv(results_lm,
  paste0("output/tables/03 lm results ", ANALYSIS LABEL, ".csv"))
```

Follow-Up Analyses Based On The Assumption Check Results:

```
# ═══════════════════════════════════════════════════════════════════════
# FOLLOW-UP ANALYSES BASED ON ASSUMPTION CHECKS
# ═══════════════════════════════════════════════════════════════════════

library(sandwich)  # robust standard errors

# ─── Follow-up 1: Robust standard errors (required if Breusch-Pagan p < 0.05) ───
if (bp_test$p.value < 0.05) {
  cat("\n─── Robust standard errors (heteroscedasticity correction) ───\n")
  library(sandwich)
  robust_results <- coeftest(model_adj, vcov = vcovHC(model_adj))
  print(robust_results)
  cat("NOTE: Use these standard errors and p-values for reporting.\n")
  cat("      Coefficients are identical — only standard errors change.\n")
}

# ─── Follow-up 2: Sensitivity analysis (remove influential observations) ─────────
if (length(influential) > 0) {
  cat("\n─── Sensitivity analysis (removing influential observations) ───\n")

  # Use the influential observations flagged by Cook's Distance above
  influential_fips <- analysis_df$fips[influential]
  cat("Removing observations with FIPS:", influential_fips, "\n")

  analysis_df_trim <- analysis_df %>%
    filter(!fips %in% influential_fips)

  model_trim <- lm(formula_adj, data = analysis_df_trim)

  cat("Trimmed model (n =", nrow(analysis_df_trim), "):\n")
  summary(model_trim)

  cat("\nComparison: key coefficients:\n")
  cat("                    Full model    Trimmed model\n")
  for (v in PREDICTOR_VARS) {
    full_coef  <- round(coef(model_adj)[v], 3)
    trim_coef  <- round(coef(model_trim)[v], 3)
    cat(sprintf("%-20s %10s    %10s\n", v, full_coef, trim_coef))
  }
  cat("\nIf coefficients are similar, results are robust to influential
observations.\n")
}
```

Logistic Regression with Full Assumption Checks

Logistic regression models the probability of a binary outcome. This script helps convert your continuous outcome variable to binary (above/below the state median), fit a logistic model, check all key assumptions, and report odds ratios.

Logistic Regression Assumptions and Diagnostics

1. **Binary outcome.** The outcome is 0/1. This script creates the binary variable automatically by dichotomizing your chosen outcome at the state median.
2. **Independence.** Observations are independent, confirmed by the study design or stated limitation in your report.
3. **No multicollinearity.** Predictors are not highly correlated with each other. Check: VIF.
4. **No complete separation.** No single predictor perfectly predicts the outcome (all 0s or all 1s for one group). This causes model failure. Check: examine frequency tables.
5. **Adequate sample size.** At least 10 events (outcome = 1) per predictor variable.
6. **Goodness of fit.** The model adequately fits the observed data. Check: Hosmer-Lemeshow test ($p > 0.05$ means good fit).

```
# ═══ PART F: Logistic Regression & Assumption Checks ═══════════════════
library(ResourceSelection)  # Hosmer-Lemeshow test

# ── Create binary outcome at the state median ──────────────────────────
state_median <- median(analysis_df[[OUTCOME_VAR]], na.rm=TRUE)
binary_outcome <- paste0(OUTCOME_VAR, "_high")

logistic_df <- analysis_df %>%
  mutate(
    !!binary_outcome := as.integer(.data[[OUTCOME_VAR]] > state_median)
  )

n_events   <- sum(logistic_df[[binary_outcome]] == 1, na.rm=TRUE)
n_nonevent <- sum(logistic_df[[binary_outcome]] == 0, na.rm=TRUE)

cat("\nBinary outcome:", binary_outcome, "\n")
cat("  Threshold (state median):", round(state_median, 2),
    "[", OUTCOME_VAR, "]\n")
cat("  High (", binary_outcome, "= 1):", n_events, "observations\n")
cat("  Low  (", binary_outcome, "= 0):", n_nonevent, "observations\n")

# ── Assumption 5 check: events-per-variable ────────────────────────────
cat("\n── Assumption 5: Sample size adequacy ──\n")
epv <- n_events / length(PREDICTOR_VARS)
cat("Events per predictor variable (EPV):", round(epv,1), "\n")
if (epv < 10) {
  cat("WARNING: EPV <10. Consider reducing predictor count to",
      floor(n_events/10), "or fewer.\n")
} else {
  cat("EPV OK (>=10). Sample size is adequate.\n")
}

# ── Unadjusted models — one per predictor ──────────────────────────────
# Standard epidemiological practice: run one unadjusted model per predictor
# Each gives a crude odds ratio (unadjusted OR)
cat("\n── Unadjusted logistic models (crude OR — one predictor each) ──\n")

unadj_results <- map_dfr(PREDICTOR_VARS, function(var) {
  formula_unadj <- as.formula(paste(binary_outcome, "~", var))
```

```
  model unadj   <- glm(formula unadj, data = logistic df,
                       family = binomial(link = "logit"))
  tidy(model unadj, conf.int = TRUE, exponentiate = TRUE) %>%
    filter(term == var) %>%
    mutate(model = "Unadjusted", predictor = var,
           across(where(is.numeric), ~round(., 4)))
})

print(unadj results %>%
        select(predictor, estimate, conf.low, conf.high, p.value))

# ── Fully adjusted logistic model — all predictors together ──────────────
cat("\n── Fully adjusted logistic model (all predictors) ──\n")
formula_logit <- as.formula(
  paste(binary outcome, "~", paste(PREDICTOR VARS, collapse=" + "))
)
logit model <- glm(formula logit, data=logistic df,
                   family=binomial(link="logit"))
summary(logit_model)

# ═══════════════════════════════════════════════════════════════════════════
# ASSUMPTION CHECKS — LOGISTIC REGRESSION
# ═══════════════════════════════════════════════════════════════════════════

cat("\n══ ASSUMPTION CHECKS: LOGISTIC REGRESSION ══\n")

# ── Assumption 3: VIF — multicollinearity ────────────────────────────────
cat("\n── Assumption 3: Multicollinearity (VIF) ──\n")
vif logit <- vif(logit model)
print(round(vif logit, 2))
if (any(vif_logit > 5)) {
  cat("WARNING: VIF > 5 — multicollinearity present. Remove correlated
predictors.\n")
} else {
  cat("VIF OK — no multicollinearity problem.\n")
}

# ── Assumption 4: Complete separation check ──────────────────────────────
cat("\n── Assumption 4: Complete separation check ──\n")
# If any predictor perfectly predicts the outcome, glm() will warn about
# 'fitted probabilities numerically 0 or 1'.
# Also check for very large coefficients (> 10 in log-odds scale).
coef_check <- coef(logit_model)[-1]   # exclude intercept
if (any(abs(coef check) > 10, na.rm=TRUE)) {
  cat("WARNING: Very large coefficients detected — possible complete
separation.\n")
  cat("Variables with large coefficients:", names(which(abs(coef check)>10)), "\n")
  cat("ACTION: Check frequency table for that predictor by outcome category.\n")
} else {
  cat("No evidence of complete separation (all log-odds coefficients < 10).\n")
}

# ── Assumption 6: Hosmer-Lemeshow goodness-of-fit test ──────────────────
cat("\n── Assumption 6: Hosmer-Lemeshow goodness-of-fit test ──\n")
cat("Null hypothesis: model fits the data adequately (p > 0.05 = good fit)\n")
hl test <- hoslem.test(
  x = logistic_df[[binary_outcome]],
```

```
  y = fitted(logit model),
  g = 10   # 10 groups (default)
)
print(hl test)
if (hl_test$p.value < 0.05) {
  cat("RESULT: p <0.05 — model may not fit well. Consider revising predictors.\n")
} else {
  cat("RESULT: p =", round(hl_test$p.value,3), "— model fits the data
adequately.\n")
}

# ── Model fit statistics ──────────────────────────────────────────────
cat("\n── Model fit statistics ──\n")
cat("AIC:", round(AIC(logit_model),2), "\n")
cat("Null deviance:",     round(logit model$null.deviance,2), "\n")
cat("Residual deviance:", round(logit_model$deviance,2), "\n")
mcfadden <- 1 - (logit model$deviance / logit model$null.deviance)
cat("McFadden pseudo-R²:", round(mcfadden,3), "\n")
cat("(0.2-0.4 = good fit for logistic models)\n")

# ── Odds Ratios (exponentiated coefficients) ──────────────────────────
cat("\n── Odds Ratios with 95% Confidence Intervals ──\n")
results logit <- tidy(logit model, conf.int=TRUE, exponentiate=TRUE) %>%
  mutate(across(where(is.numeric), ~round(.,4)))
print(results_logit)

# ── Classification performance ────────────────────────────────────────
cat("\n── Classification performance (threshold = 0.5) ──\n")
predicted class <- as.integer(fitted(logit model) >= 0.5)
actual class    <- logistic df[[binary outcome]]
conf_matrix     <- table(Predicted=predicted_class, Actual=actual_class)
print(conf matrix)

accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)
cat("Accuracy:", round(accuracy*100,1), "% \n")

# ── Save results ──────────────────────────────────────────────────────
write csv(results logit,
  paste0("output/tables/04 logit OR ", ANALYSIS LABEL, ".csv"))
cat("Script #7 complete.\n")
```

Assumption Check Summary Table

Use this table to document the result of every assumption check after running Script #7. Record your findings before reporting results.

Assumption	Test / Method	Pass criteria	Your result	Action if violated
LINEAR REGRESSION				
Linearity	Residuals vs Fitted plot	Random scatter, no curve	[ ] Pass [ ] Fail	Log-transform the outcome or use polynomial terms
Homoscedasticity	Breusch-Pagan test, Scale-Location plot	$p > 0.05$, flat red line	[ ] Pass [ ] Fail	Use heteroscedasticity-robust standard errors (sandwich package)

Normality of residuals	Shapiro-Wilk test, Q-Q plot	$p > 0.05$, points on diagonal	[ ] Pass [ ] Fail	Log-transform outcome
No influential outliers	Cook's Distance	No observations above 4/n threshold	[ ] Pass [ ] Fail	Run sensitivity analysis with and without flagged observations
No multicollinearity	VIF (all predictors)	All VIF < 5	[ ] Pass [ ] Fail	Remove highest VIF predictor and re-run
LOGISTIC REGRESSION				
Sample size (EPV)	Events per variable calculation	EPV ≥ 10	[ ] Pass [ ] Fail	Reduce number of predictors
No complete separation	Large coefficient check	No log-odds > 10	[ ] Pass [ ] Fail	Remove or combine the separating predictor
Goodness of fit	Hosmer-Lemeshow test	$p > 0.05$	[ ] Pass [ ] Fail	Revise model by adding or removing predictors
No multicollinearity	VIF (same as linear)	All VIF < 5	[ ] Pass [ ] Fail	Remove highest VIF predictor and re-run
BOTH REGRESSIONS				
Independence of observations	Study design review; check for spatial clustering if relevant	Each geographic area represents one independent observation; no obvious duplicates or nested observations	[ ] Pass [ ] Fail	Spatial autocorrelation: consider Moran's I if needed

5. Export and Data Dictionary

At the end of your analysis workflow, make sure these files exist in your output/ folder before moving to Part 3.

Required Output Files

Folder	File	Description
DATA FOLDER		
data/clean/	01_acs_clean.csv	ACS 5-year cleaned county data
	02_places_clean.csv	CDC PLACES health outcomes
	03a_svi_clean.csv	SVI 2022 county data
	03b_eji_county_clean.csv	EJI 2024 aggregated to county
	04_hpsa_clean.csv	HRSA HPSA shortage designations
	05a_usda_food_access_clean.csv	USDA food access atlas
	05b_chr_clean.csv	County Health Rankings 2025
	05c_nces_ccd_clean.csv	NCES CCD enrollment
data/master/	sdoh_[state]_master.csv	Final merged master dataset (Script #6)

OUTPUT FOLDER		
output/figures/	02_correlation_[ANALYSIS_LABEL].png	Correlation matrix plot
	03_lm_diagnostics_[ANALYSIS_LABEL].png	Linear regression diagnostic plots
output/tableau/	sdoh_[state]_tableau.csv	Master dataset for Tableau
output/tables/	01_descriptive_[ANALYSIS_LABEL].csv	Descriptive statistics
	03_lm_results_[ANALYSIS_LABEL].csv	Linear regression coefficients and CIs
	03_lm_results_[ANALYSIS_LABEL].csv	Robust standard errors results, if required by the Breusch-Pagan test
	04_logit_OR_[ANALYSIS_LABEL].csv	Logistic regression odds ratios
DICTIONARY FOLDER		
dictionary/	data_dictionary.csv	Variable labels, sources, and years
ROOT FOLDER		
sdoh_toolkit/	README.txt	Project documentation

Data Dictionary Template
Every project must include a data dictionary listing every variable in the master CSV. This is required for your GitHub repository and for open scholarship compliance.

Generating Your Project README
The README code automatically generates a plain-text documentation file that records everything needed to reproduce your analysis (e.g., who ran it, when, which data were used, which variables were selected, which software versions were installed, and where the output files are saved). This is a core open scholarship requirement for any publicly shared research project.

```
# ── Generate README.txt automatically ──────────────────────────────────────
sink("README.txt")
cat("=== SDOH TOOLKIT — PROJECT README ===\n\n")
cat("Author:           [Your name]\n")
cat("Institution:      [Your institution]\n")
cat("Date completed:   ", format(Sys.Date(), "%Y-%m-%d"), "\n")
cat("Research question:[One sentence]\n\n")

cat("--- Analysis configuration ---\n")
cat("State:            ", STATE_LABEL, "\n")
cat("Geography:        County level\n")
cat("Outcome variable: ", OUTCOME_VAR, "\n")
cat("Predictors:       ", paste(PREDICTOR_VARS, collapse=", "), "\n")
cat("Analysis label:   ", ANALYSIS_LABEL, "\n\n")

cat("--- Script run order ---\n")
cat("01_acs.R → 02_places.R → 03_svi_eji.R → 04_hrsa.R\n")
cat("→ 05 usda nces chr.R → 06 merge all.R → 07 analysis.R\n\n")

cat("--- Data download dates ---\n")
cat("ACS 2020-2024:    [date]\n")
cat("CDC PLACES 2025:  [date]\n")
cat("SVI 2022:         [date]\n")
cat("EJI 2024:         [date]\n")
cat("HRSA AHRF 2024-25:[date]\n")
cat("USDA Atlas 2019:  [date]\n")
cat("CHR&R 2025:       [date]\n")
cat("NCES CCD 2024-25: [date]\n\n")
```

```
cat("--- Output files ---\n")
cat("Master CSV:    data/master/sdoh_", STATE_LABEL, "_master.csv\n")
cat("Tableau CSV:   output/tableau/sdoh ", STATE_LABEL, " tableau.csv\n")
cat("Descriptive:   output/tables/01 descriptive ", ANALYSIS_LABEL, ".csv\n")
cat("LM results:    output/tables/03_lm_results_", ANALYSIS_LABEL, ".csv\n")
cat("Logit results: output/tables/04 logit OR ", ANALYSIS_LABEL, ".csv\n")
cat("Correlation:   output/figures/02 correlation ", ANALYSIS_LABEL, ".png\n")
cat("Diagnostics:   output/figures/03_lm_diagnostics_", ANALYSIS_LABEL, ".png\n\n")

cat("--- GitHub repository ---\n")
cat("Repository URL: [your GitHub URL]\n")
cat("Visibility:     Public\n\n")

cat("--- R environment ---\n")
cat("R version:", R.version$major, ".", R.version$minor, "\n")
cat("OS:", Sys.info()["sysname"], Sys.info()["release"], "\n\n")

cat("--- Package versions ---\n")
pkgs <- c("tidycensus","tidyverse","haven","readxl",
          "janitor","skimr","corrplot","car","broom","lmtest",
          "ResourceSelection","sandwich","devtools")
for(p in pkgs){
  if(requireNamespace(p, quietly=TRUE)){
    cat(p, ":", as.character(packageVersion(p)), "\n")
  }
}
sink()
cat("README.txt saved.\n")
```

Create `dictionary/data_dictionary.csv`

Every project must include a data dictionary listing every variable in the master CSV. This is required for your GitHub repository and for open scholarship compliance. Run this code after Script #6 has completed successfully.

```
# ── Create data dictionary from master file ──────────────────────────────

library(tidyverse)

master <- read_csv("data/master/sdoh_il_master.csv")

# ── Base dictionary from column names ────────────────────────────────────
data_dict <- tibble(
variable name = names(master),
label = c(
"5-digit county FIPS code (character)",          # fips
"County name (ACS format, includes state)",       # county name
"Poverty rate (% persons below poverty line)",    # poverty_rate
"Median household income ($)",                    # median hh income
"Unemployment rate (% civilian labor force)",     # unemployment rate
"SNAP participation rate (% households)",         # snap_rate
"Rent cost burden rate (% renters paying 30%+ income on rent)", # rent burden rate
"% adults 25+ with less than HS diploma",         # pct no hs
"% adults 25+ with HS diploma or equivalent",     # pct_hs_grad
"% adults 25+ with bachelor's degree or higher", # pct bachelor plus
"Uninsured rate (% population, ACS-derived)",     # uninsured rate
"Adult obesity prevalence (%, CDC PLACES 2025)", # obesity_pct
```

```
"Depression prevalence (%, CDC PLACES 2025)",    # depression pct
"Diagnosed diabetes prevalence (%, CDC PLACES 2025)", # diabetes_pct
"COPD prevalence (%, CDC PLACES 2025)",          # copd pct
"Current smoking prevalence (%, CDC PLACES 2025)", # smoking pct
"Mammography use rate (%, CDC PLACES 2025)",     # mammography_pct
"Physical inactivity prevalence (%, CDC PLACES 2025)", # inactivity pct
"Current asthma prevalence (%, CDC PLACES 2025)", # asthma pct
"Uninsured % (CDC PLACES 2025, Access2 measure)", # uninsured_places
"High blood pressure prevalence (%, CDC PLACES 2025)", # hypertension pct
"Colorectal cancer screening rate (%, CDC PLACES 2025)", # colorectal screen pct
"SVI overall percentile (0-1; higher=more vulnerable, CDC/ATSDR 2022)", #
svi overall
"SVI Theme 1: socioeconomic status percentile",  # svi socioeco
"SVI Theme 2: household characteristics percentile", # svi_hh_char
"SVI Theme 3: racial/ethnic minority and language percentile", # svi minority
"SVI Theme 4: housing type and transportation percentile", # svi_housing
"% below 150% poverty (SVI component)",          # pct poverty
"% unemployed (SVI component)",                  # pct unemp svi
"% uninsured (SVI component)",                   # pct_uninsured_svi
"EJI overall county mean percentile (CDC/ATSDR 2024)", # eji overall
"EJI Environmental Burden mean percentile",      # eji env burden
"EJI Social Vulnerability mean percentile",      # eji_social_vuln
"EJI Health Vulnerability mean percentile",      # eji health vuln
"Number of census tracts aggregated for EJI",    # n tracts
"Primary care shortage designation (0=none, 1=partial, 2=full)", # hpsa_prim_care
"Dental shortage designation (0=none, 1=partial, 2=full)", # hpsa dental
"Mental health shortage designation (0=none, 1=partial, 2=full)", #
hpsa_mental_health
"Any shortage designation (1=yes, 0=no)",        # hpsa any shortage
"Primary care shortage binary (1=yes, 0=no)",    # hpsa prim care designated
"% census tracts that are low-income AND low-access (USDA)", # pct_lila_tracts
"Mean food access share (USDA)",                 # mean dist supermarket
"% housing units lacking vehicle and far from supermarket", # pct no vehicle far
"Premature death rate (CHR&R 2025)",             # premature_death
"Avg physically unhealthy days per month (CHR&R 2025)", # poor health days
"Avg mentally unhealthy days per month (CHR&R 2025)", # poor mental days
"Social associations per 100,000 population (CHR&R 2025)", # social_associations
"Voter turnout % (CHR&R 2025, presidential election)", # voter turnout
"Adult smoking % (CHR&R 2025)",                  # adult smoking
"Adult obesity % (CHR&R 2025)",                  # adult_obesity
"Food insecurity % (CHR&R 2025)",                # food insecurity
"Number of school districts in county (NCES CCD)", # n districts
"Total student enrollment in county",            # total_enrollment
"% students eligible for free lunch",            # pct free lunch
"% students eligible for reduced-price lunch"    # pct reduced lunch
),
  source = c(
    "U.S. Census Bureau", "U.S. Census Bureau",
    rep("ACS 5-Year 2020-2024", 9),
    rep("CDC PLACES 2025", 11),
    rep("CDC/ATSDR SVI 2022", 8),
    rep("CDC/ATSDR EJI 2024", 5),
    rep("HRSA AHRF 2025", 5),
    rep("USDA Food Access Atlas 2019", 3),
    rep("CHR&R 2025", 8),
    rep("NCES CCD 2024", 4)
  ),
```

```
  year = c(
    "2024", "2024",
    rep("2020-2024", 9),
    rep("2021-2022 BRFSS", 11),
    rep("2022", 8),
    rep("2024", 5),
    rep("2025", 5),      #HPSA
    rep("2019", 3),      # USDA
    rep("2025", 8),      # CHR&R
    rep("2024", 4)       # NCES
  )
)

write_csv(data_dict, "dictionary/data_dictionary.csv")
cat("Data dictionary saved with", nrow(data dict), "variables.\n")
```

6. Part 2 Checklist

Complete all items before moving to Part 3. Items 1-5 are required. Items 6 and above are best practices for open scholarship.

- ☐ I have installed R 4.3+, RStudio, and all required packages (Section 1B).
- ☐ I have obtained a Census API key and stored it with `census_api_key()` (Section 1C).
- ☐ I have created the project folder structure and all required subdirectories (Section 1D).
- ☐ Script #1 runs without errors and produces 01_acs_clean.csv with the expected number of rows for your chosen geography (e.g., 102 for IL counties, approximately 3,000 for IL census tracts).
- ☐ Scripts #2-#5 run without errors and produce clean CSV files in data/clean/.
- ☐ Script #6 produces `sdoh_[state]_master.csv` with the expected number of rows for your chosen geography and no rows dropped during merges.
- ☐ Script #7 produces descriptive statistics, correlation matrix, unadjusted and adjusted regression results, assumption check outputs, and follow-up analyses where required.
- ☐ I have reviewed VIF values and confirmed no multicollinearity problem (VIF < 5 for all predictors).
- ☐ I have completed the Assumption Check Summary Table and documented Pass/Fail for all linear and logistic regression assumptions.
- ☐ If Breusch-Pagan $p < 0.05$, I have applied robust standard errors using the sandwich package.
- ☐ If Cook's Distance flagged influential observations, I have run and reviewed the sensitivity analysis.
- ☐ `output/tableau/sdoh_[state]_tableau.csv` exists and is ready for Tableau (Part 3).
- ☐ `dictionary/data_dictionary.csv` exists and lists all variables with source and year.
- ☐ README.txt has been generated and saved to the project root folder. GitHub repository setup and open sharing will be completed in Part 3. If this toolkit is being used as part of a course, submit the repository URL according to your instructor's directions.

If any script produces an error, check: (1) file paths match your folder structure exactly, (2) FIPS columns are 5-digit character strings, (3) package versions are current — run `update.packages()`, and (4) your Census API key is loaded. Run `readRenviron("~/.Renviron")` if `sys.getenv("CENSUS_API_KEY")` returns blank. Do not move to Part 3 until the master CSV has the expected number of rows for your chosen geography with no critical missing data.

Ready for the next step?

Part 3 covers building and publishing your Tableau Public dashboard as an open, freely accessible resource.

Part 3: Tableau, Repository & Open Sharing →

Part 3. Tableau, Repository & Open Sharing

Build and publish your Tableau Public dashboard as a publicly accessible open resource.

Prerequisite: Parts 1 & 2 complete | **Level:** Beginner–Intermediate | **Time:** ~3–4 hours

Learning Objectives

- Connect your master CSV to Tableau and build interactive visualizations for SDOH variables.
- Apply dashboard design principles to create a multi-view interactive SDOH dashboard.
- Publish your completed dashboard to Tableau Public as an openly accessible resource.
- Document and share your project through GitHub, create a citable research record with Zenodo when appropriate, and explore institutional repository options for open sharing.

What you need before starting:

- Sign up for a free Tableau Public account at public.tableau.com (takes 2 minutes).
- Your master file: output/tableau/sdoh_[state]_tableau.csv from Part 2.
- A modern browser: Chrome or Edge is recommended. Firefox and Safari also work.
- No software download required.

Geographic Unit Used In This Part

Part 2 demonstrates the complete data workflow at the county level using Illinois as the example state. Part 3 uses the same county-level master CSV as its input. All map types, filter formulas, and step-by-step instructions in this part use county as the geographic unit. Students who adapted Scripts in Part 2 for census tract or ZCTA geography can apply the same Tableau steps but will need to adjust the geographic role assignment and FIPS filter formula accordingly.

1. Connect Your Data to Tableau

Corresponding Videos: Access Tableau & Connect Your Data

Access Your Free Tableau Public Account

1. Create or sign in to your Tableau Public account
    - First-time user: Go to public.tableau.com and click **Sign Up** in the top right corner. Enter your name, email, and password. Then check your email and click the confirmation link to activate your account.
    - Returning user: Go to public.tableau.com and click **Sign In** in the top right corner. Enter your email and password.
2. After signing in, click your name in the top-right corner, then select **My Profile**. All your published dashboards will appear here.
3. On your profile page, click the blue **Create a Viz** button. This opens the Tableau Public web interface in your browser.

Upload Your CSV and Connect Data

1. The Connect to Data panel opens automatically when you click Create a Viz. Under Files, click Upload from Computer.

2. Navigate to `output/tableau/sdoh_[state]_tableau.csv`, then click Open. Tableau uploads and previews the file.
3. Tableau will display a preview grid of your data rows. Scroll horizontally to verify that all structural tracking columns are present (`fips`, `county`, `svi_overall`, etc.).
4. Locate the FIPS column in the preview. If a numeric symbol (#) appears above the column name, click it and select **Change Data Type** to **String (Abc)**. FIPS must be text for mapping to work correctly. If you skip this step, the map will not be rendered.
5. Once configured, click **Update Now** and click the Worksheet 1 tab at the bottom to enter the visualization canvas.

Google Drive Alternative
If your CSV is stored in Google Drive, click Google Drive in the Connect panel and sign in. Navigate to your CSV and select it. To update your dashboard with new data later, click Data, then Refresh Data Source in Tableau and republish.

Latitude and Longitude
When you assign the FIPS geographic role, Tableau automatically generates `Latitude (generated)` and `Longitude (generated)` from your FIPS column. These fields mark the county's centroid, which works for filled maps or bubble charts but not for detailed address mapping. For street-level detail, use `tidygeocoder` in R to create precise coordinate columns.

2. Choose Your Map Type

Four Visualization Categories for Regional Data Variance Mapping
When translating SDOH into geographic maps, your choices are driven by data structure and communication goals. The Spatial Visualization Product Matrix below breaks down four common visualization products along two dimensions: Map Typology Focus and System Complexity.

Map Typology Focus refers to what the map emphasizes: specific locations, spatial patterns, or both.
- Reference Map: Where things are (locations, addresses, specific geographic centers). Example: Where are the regional resource centers located?
- Thematic Map: How values vary across space (patterns, rates, statistical differences). Example: Which geographic units have the highest diabetes prevalence?

System Complexity refers to how much data integration, interactivity, and coordination are built into the final product.
- Low Complexity: One dataset with simple interactions. A user can click an isolated point or boundary to see its name, but there is no active filtering. Only one variable is shown at a time.
- High Complexity: Multiple datasets across coordinated views. Filtering one chart automatically updates all other sheets simultaneously using variable selectors and linked highlighting.

(Source: Healthy Regions & Policies Lab)

Spatial Visualization Product Matrix

	Asset map	**Thematic map**	**Dashboard**	**Story map**
Map Type Focus	Reference Mapping	Thematic Mapping	Thematic, Multivariate	Reference and Thematic (narrative sequence)
System Complexity	Low	Low to Medium	High Complexity	Medium to High

Tableau Component	Point/Shape Marks	Filled map (polygon) Bubble map (circle)	Coordinated Multi-Sheet Canvas	Built-in Tableau Story Tab
Data Structure	CSV with Latitude and Longitude	CSV merged to FIPS	Master CSV or relational data source with multiple variables	Master CSV, arranged into narrative sequence
Primary Analytical Goal	Documenting health infrastructure inventory and allocation	Showing how an SDOH variable varies across regions	Exploring multiple SDOH variables simultaneously	Walking an audience through a guided narrative
Toolkit Alignment	HRSA HPSA facility or location layer	County choropleth	The 3-view Tableau dashboard	Final presentation layer linking findings

Dashboard and Story Map are containers as they do not have their own map type. You build Asset Maps and Thematic Maps first (Sections 3 and 4), then assemble them into a Dashboard (Section 5) or Story Map (Section 6).

Map Types Buildable from Your Master CSV

All six map types below can be built directly from `sdoh_[state]_tableau.csv`. Tableau generates latitude and longitude automatically from your FIPS column.

Map Type	Asset Map	Thematic Map	Dashboard / Story	Variables Needed from Master CSV
Simple Map	✓		✓	`fips`, `county_name`
Point Distribution Map	✓		✓	`fips` (Tableau generates lat/lon automatically)
Filled (Choropleth) Map		✓	✓	`fips`, & any of your numeric variables
Proportional Symbol Map		✓	✓	`fips`, & Count Variable (e.g., `total_enrollment`)
Dual-Axis (Layered) Map		✓	✓	Two layers from the same dataset
Filled Map with Pie Charts		✓	✓	`fips`, & `hpsa_prim_care`, `hpsa_mental_health` (reshaped in R)

Map Type Decision Guide

If your research question is...	Use this map type	Category
Which counties have the highest diabetes/poverty/SVI?	Filled (Choropleth) Map	Thematic Map
Where are health care shortage areas located?	Point Distribution Map	Asset Map
How severe are shortages per county?	Proportional Symbol Map	Thematic Map
Which counties have shortages and high diabetes?	Dual-Axis (Layered) Map	Thematic Map
What proportion of shortages are mental health?	Filled Map with Pie Charts	Thematic Map
What is my study area? (reference only)	Simple Map	Asset Map
I want to show multiple SDOH views together	Dashboard	Dashboard
I want to walk through the findings step by step	Story Map	Story Map

Data Structures (polygon vs. point) and Build Steps

The most fundamental choice in Tableau mapping is whether your data uses area boundaries (polygon data) or point locations. Polygon data uses a geographic identifier like FIPS code. Tableau draws the county boundary automatically. Point data requires Latitude and Longitude columns in decimal degrees. You cannot make a choropleth from point data. Tableau can use recognized geographic identifiers such as county FIPS codes to generate centroid points. Facility- or address-level point mapping requires explicit latitude and longitude.

Maps using area boundaries (polygon data)	**Maps using point data (lat/lon coordinates)**
• Simple map — reference overview of locations • Filled (choropleth) map — ratio or aggregated data by area • Filled map with pie charts — composition within each area	• Point distribution map — locations and visual clusters • Proportional symbol map — quantitative values at locations • Heatmap (density map) — concentration and trends • Flow (path) map — movement over time • Origin–destination (spider) map — connections between places • Dual-axis (layered) map — two map types combined

Note: Heatmap, Flow map, and Origin-Destination map cannot be generated from the master CSV in this toolkit. They require individual point data, time-series data, or origin-destination pairs respectively.

3. Asset Maps

Corresponding Video: All Maps (Asset Maps)

Asset maps show where community resources or geographic units are located. They use reference mapping. No data values are encoded in color. Use asset maps to orient your audience to the study area or show where specific resources are located.

Simple Map

A reference map that displays all counties without encoding any data variable in color. Shows where things are. Use it to orient a non-specialist audience to your study area before showing data.

Data Structure Needed

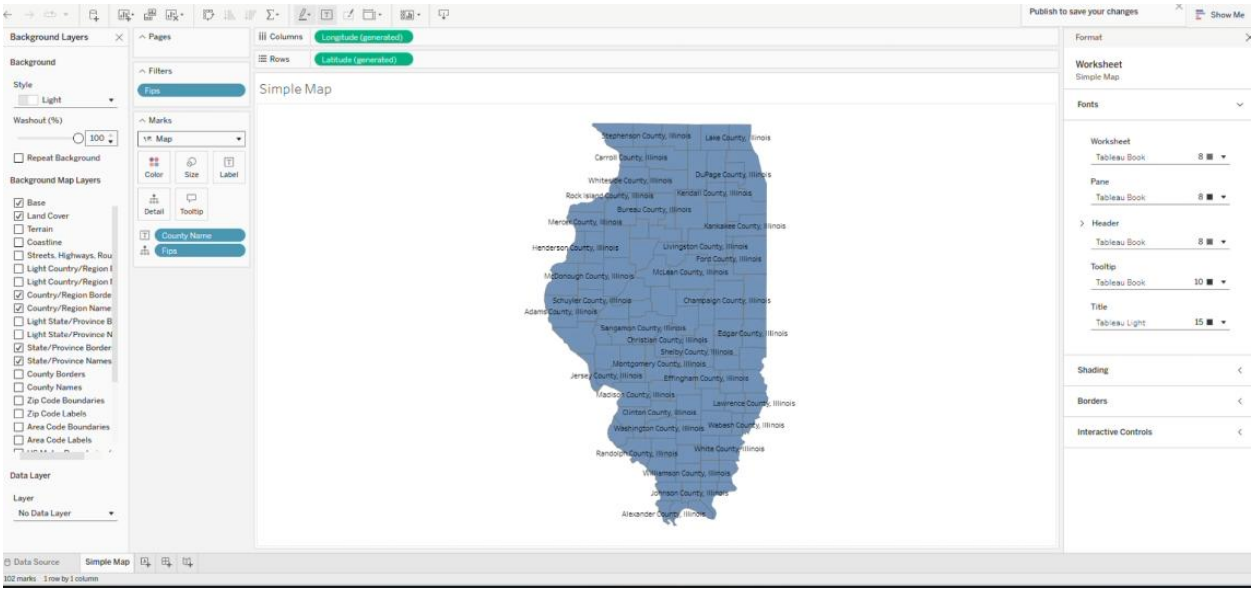
Your master CSV already has this. The column, `fips`, is required:

```
fips   county_name
17001  Adams County, Illinois
17003  Alexander County, Illinois
17005  Bond County, Illinois
...         ...
Required: fips (String, 5-digit)
Optional: county_name (for labels)
```

Step-By-Step: Simple Map

1. Right-click the Worksheet 1 tab and rename it: Simple Map.
2. Right-click the FIPS field in the Data pane, then **Geographic Role**, then **County**. A globe icon shows. Tableau automatically creates Latitude (generated) and Longitude (generated).
3. Double-click the FIPS field. Tableau generates an Illinois map with centroid dots in each county.
4. Drag Fips to the Filters. The filter window will automatically pop up. Click **Condition**, select **By formula**, and enter: `LEFT([Fips], 2) = '17' for Illinois`. Click OK.

5. Under the Marks card dropdown menu, change the Mark type from *Automatic* to **Map**. The map should show IL counties in blue.
6. Drag `County Name` to the **Label** shelf.
7. Go to the top menu bar, click **Format → Worksheet** to change the font size to 8pt.
8. Go to the top menu bar, click **Map → Background Layers**. In the left pane, set **Washout** to 100%.



Point Distribution Map

A point distribution map plots the locations of entities as individual dots, allowing you to spot visual clusters or regional geographic patterns of occurrence. In public health research, this technique is used to map regional health care shortages, show the distribution of health centers or food pantries, identify spatial clustering of resource deserts in rural areas, and plot educational access points for regional analysis. This toolkit supports two data sources for point distribution maps. Try both to compare what each reveals about your study area.

Data Source 1: County Centroids (Master CSV)

Individual dots indicate the locations of each geographic unit's centroid. Tableau automatically generates coordinates from your FIPS column. Use this map to show where counties with specific characteristics are located.

Data Structure Needed

The column `Fips` is required with any variable (e.g., poverty rate) for color or size.

```
name             latitude(generated)    longitude(generated)    poverty rate
Adams County           39.9217                -91.1849              14.5
Alexander Co.          37.1481                -89.3201              22.0
Bond County            38.8922                -89.4323               8.0
...                       ...                    ...                  ...
Required: Latitude (decimal, Measure)
Required: Longitude (decimal, Measure)
Optional: any variable for Color or Size
```

Step-By-Step: County Centroids

1. Click the Sheet tab and rename it: Point Distribution Map.
2. Double-click the `FIPS` field. Tableau will automatically generate a U.S. map with one centroid dot per county.
3. Drag FIPS to the Filters shelf. Click the **Condition** tab, select By formula, and enter: `LEFT([Fips],2) = '17'`. Click OK.
4. In the Marks card dropdown menu, change the mark type from *Automatic* to **Circle**.
5. Drag `County Name` from the Data pane and drop it onto the **Detail** tile on the Marks card. This makes the field available for your tooltip.
6. Drag `poverty_rate` (or any SDOH variable) to the **Color** shelf. Click **Color**, then set Opacity to 70%.
7. Click the **Tooltip**. Delete the default text in the editor box. Use the **Insert** dropdown menu in the top-right corner of the window to add your fields, structuring the text like this: `County: <County Name> | Poverty: <SUM(Poverty_Rate)>%`

Data Source 2: Actual Health Center Locations (HRSA File)
Individual dots mark HRSA-funded health center sites with precise street-level coordinates. Each dot represents one health center. Best for showing where specific facilities are located within and across counties.

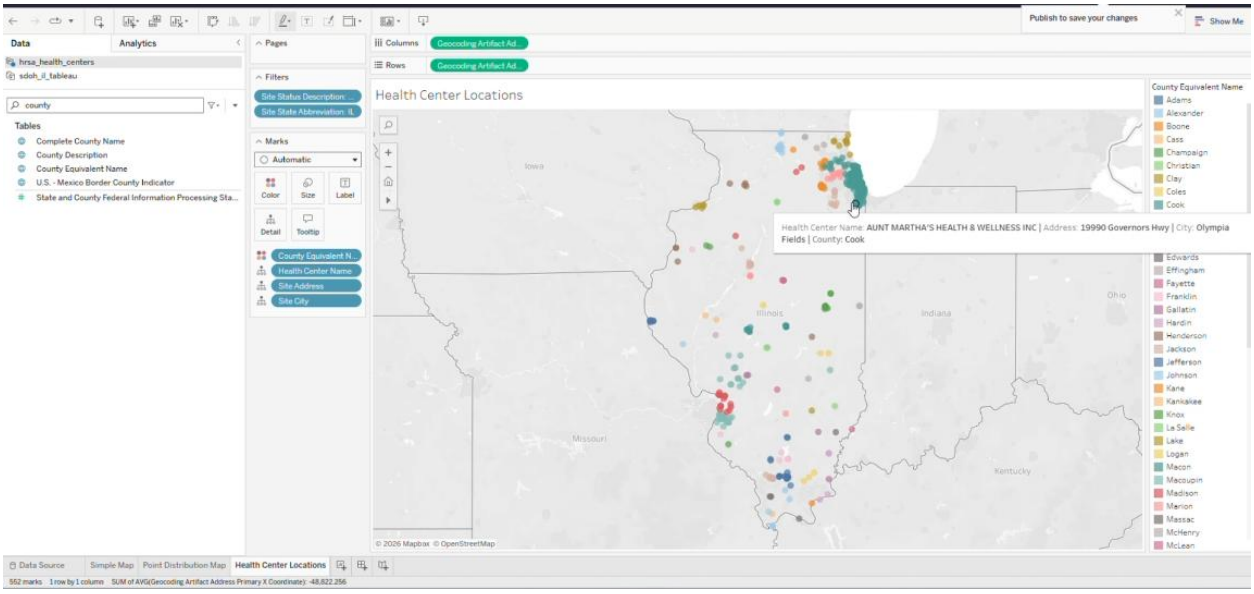
Download Before Building:
Go to data.hrsa.gov/data/download and scroll to **Health Center Service Delivery and Look-Alike Sites**. Download the CSV (updated daily, free, no login required). Save as data/raw/hrsa_health_centers.csv. This file already contains latitude and longitude, thus upload it directly to Tableau as a second data source. No R processing required.

Data Structure Needed
Latitude (Measure) and longitude (Measure) are required.

site name	city	county	state abbreviation	latitude	longitude
Heartland Health	Peoria	Peoria County	IL	40.6936	-89.5890
ACCESS Community	Chicago	Cook County	IL	41.8781	-87.6298

Step-by-Step: Actual Health Center Locations
1. In Tableau, click the **Data** menu, then **New Data Source**, then **Upload from Computer**. Upload `hrsa_health_centers.csv`. Tableau loads the full national file.
2. In the Data pane, right-click `Geocoding Artifact Address Primary X Coordinate`, hover over **Geographic Role**, and select **Longitude**. Then, right-click `Geocoding Artifact Address Primary Y Coordinate`, hover over **Geographic Role**, and select **Latitude**. (This tells Tableau to load the real U.S. base map).
3. Click **Update Now** to complete the data load.
4. Click the Worksheet tab + to open a new sheet. Rename it: Health Center Locations.
5. In the top-left of the Data pane, make sure `hrsa_health_centers.csv` is selected.
6. Drag `Geocoding Artifact Address Primary X Coordinate` to **Columns**. Drag `Geocoding Artifact Address Primary Y Coordinate` to **Rows**. Tableau switches to a map view with one dot per health center site across the entire United States.
7. Go to the top menu bar, click **Analysis**, and uncheck **Aggregate Measures**. This forces Tableau to display every individual health center point instead of adding them together on a blank grid.
8. Drag `Site Status Description` to the Filters shelf. Select **Active**. Click OK.
9. Drag `Site State Abbreviation` to the Filters shelf. Select **IL (or your state abbreviation)**. Click OK. The map zooms to show only your state's active health centers.

10. In the Marks card dropdown menu, change the mark type from *Automatic* to **Circle**.
11. Drag `Health Center Name`, `Site Address`, and `Site City` from the Data pane onto the **Detail** tile of the Marks card so they are accessible in your tooltip text.
12. Drag `County Equivalent Name` to the Color shelf to distinguish sites by county. Click the Color card and set the Opacity slider to 70%.
13. Click the **Tooltip** card. Delete the default text in the editor window. Use the **Insert** dropdown in the top-right corner to add your loaded fields, formatting the line to read: `Site: <Health Center Name> | Address: <Site Address> | City: <Site City> | County: <County Equivalent Name>`



If you have a CSV with street addresses for facilities like food pantries, community clinics, or social service sites, you can convert those addresses to latitude and longitude coordinates using the `tidygeocoder` package in R. Once geocoded, upload the resulting CSV to Tableau as a second data source and follow the same build steps as Data Source (HRSA File) above.

```
# install.packages("tidygeocoder")
library(tidygeocoder)
library(tidyverse)

# Load your facility file with a street address column
my facilities <- read csv("data/raw/my facilities.csv")

# Geocode using free OpenStreetMap service — no API key needed
# Change 'street address' to match your actual column name
my_facilities_geo <- my_facilities %>%
  mutate(full address = paste(street address, city, state, "USA")) %>%
  geocode(full address,
          method = "osm",
          lat = latitude,
          long = longitude)

cat("Geocoded:", nrow(my facilities geo), "rows\n")
cat("Missing coordinates:", sum(is.na(my facilities geo$latitude)), "\n")

write_csv(my_facilities_geo, "output/tableau/my_facilities_geocoded.csv")
```

4. Thematic Maps

Corresponding Video: All Maps (Thematic Maps)

Thematic maps show how data values vary across geographic areas. They are the most common type of map in SDOH research. Each map type encodes data differently. You need to choose based on your research question and data structure.

Filled (Choropleth) Map

A choropleth map shades areas by data value, with darker tints indicating higher or lower concentrations. It is most common in public health research. This visualization shows how variables such as diabetes prevalence, poverty rates, or SVI values vary across counties simultaneously, helping identify geographic clusters of high or low health burden, compare county metrics to the state average, and map any continuous percentage, index, or ranked variable.

Data Structure Needed

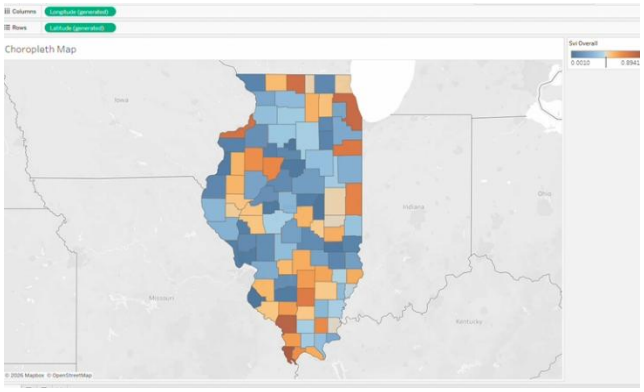
Your master CSV. Two minimum columns, including `fips` and any numeric variables

```
fips     diabetes pct    poverty rate    svi overall
17001        11.2          14.3          0.62
17003        17.8          29.7          0.91
17005        10.4          11.2          0.44
...           ...           ...           ...

Required: fips (String, 5-digit)
Required: any numeric measure variable
```

Choosing The Right Color Palette

- Sequential (single-color ramp): Use for variables that move from lower to higher values, such as poverty rates, SVI percentiles, or EJI percentiles. Example: poverty_rate (white to dark blue). In Tableau: Edit Colors, then choose a sequential palette such as Blue Sequential.
- Diverging (two-color ramp): Use when a variable has a meaningful midpoint or reference value and you want to show values above and below that point. In Tableau: Edit Colors, then choose an appropriate diverging palette.
- Never use rainbow: Rainbow color scales are not accessible for many colorblind users and do not convey order clearly. Use a sequential or diverging palette appropriate for the variable.

Step-by-step: Filled (Choropleth) Map



1. Double-click the sheet tab and rename it: Choropleth Map.
2. Right-click FIPS in the Data pane, hover over **Geographic Role**, and select **County**. A globe icon will appear next to the field.
3. Double-click the FIPS field. Tableau will generate a U.S. map displaying individual centroid dots.
4. Drag FIPS to the Filters shelf. In the pop-up prompt, select All values and click Next. Click the **Condition** tab, select **By formula**, and enter: `LEFT([Fips], 2) = '17'` for Illinois. Click OK to zoom into your state.
5. Change the mark type dropdown on the **Marks card** from *Automatic* to **Map**. The dots will instantly transform into solid, filled county shapes.
6. Drag your variable (e.g., `svi_overall`) from the Data pane and drop it directly onto the **Color** shelf on the **Marks card**. The counties will immediately shade by value.
7. Click the **Color** card and select **Edit Colors**. Choose Blue Sequential for burden/risk variables, or Orange-Blue Diverging for index variables (like SVI or EJI). Click OK. (Note: The web interface automatically syncs to your data's true minimum and maximum values).
8. Drag `County Name` and `your variable` from the Data pane and drop them onto the **Detail** tile of the Marks card. This loads them into background memory so your tooltips can read them.
9. Click the **Tooltip card**. Delete the default text in the editor window. Use the **Insert** dropdown menu in the top-right corner to add your loaded fields, formatting the text to read:
   `County: <County Name>  | Variable Name: <SUM(Variable)>`
10. Verify that all counties are shaded, no areas show Null or gray, the color legend shows the correct range, and the tooltip works on hover.

If your map shows dots instead of shaded counties, click the mark type dropdown in the Marks card and change it to Map. If the Map is unavailable, return to the Data Source tab, confirm that FIPS is a string, and reassign the County geographic role.

Proportional Symbol Map

A proportional symbol map uses circles scaled to data values at each location to show both position and magnitude simultaneously. It reveals where issues exist and their significance, ideal when absolute size matters over rate. In public health, it's used for counts of shortages, uninsured individuals, or food pantry visits and social service referrals.

Data Structure Needed

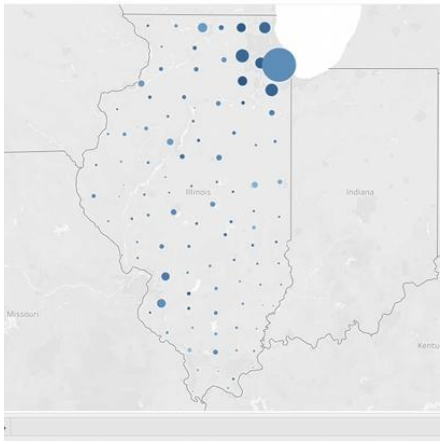
From your master CSV file, three variables are needed (1) `fips` (Tableau generates lat/lon automatically), (2) count or total variable for the Size shelf (e.g., `total_enrollment`), and (3) rate variable for the Color shelf (e.g., `poverty_rate`).

```
name         latitude(generated) longitude(generated) total_enrollment poverty_rate
Adams County       39.9217            -91.1849               330            14.5
Alexander Co.      37.1481            -89.3201               502            22.0
Bond County        38.8922            -89.4323               135             8.0

Required: Latitude, Longitude (Measures)
Required: count/total variable for Size shelf
```

```
Optional: rate or score variable for Color shelf
Key distinction: Size = count, Color = rate
```

Step-by-step: Proportional Symbol Map

1. Double-click the sheet tab and rename it: Proportional Symbol Map.
2. Right-click FIPS in the Data pane, hover over **Geographic Role**, and select **County**. A globe icon will appear next to the field.
3. Double-click the FIPS field. Tableau will automatically generate a U.S. map layout with individual centroid dots.
4. Drag FIPS to the **Filters** shelf. In the pop-up window, click the **Condition** tab, select **By formula**, and enter: `LEFT([Fips], 2) = '17'` for Illinois. Click OK to zoom into your state.
5. Change the mark type dropdown on the **Marks card** from *Automatic* to **Circle**.
6. Drag your `count variable (e.g., total_enrollment)` from the Data pane and drop it onto the **Size** shelf on the Marks card. The circles will immediately scale proportionally to the data values.
7. Drag your `rate variable (e.g., poverty_rate)` from the Data pane and drop it onto the **Color** shelf on the **Marks card**. Your map now displays magnitude by size and intensity by color.
8. Drag `County Name` from the Data pane and drop it onto the Detail tile of the Marks card so it is available in your tooltip memory.
9. Click the **Size card**. Adjust the slider slightly to the right to make the circles larger, ensuring the smallest circles remain clearly visible and the largest circles do not completely obscure the map background.
10. Right-click the title of your size legend on the right side of the screen and select **Edit Title**. Change it to match your count field (e.g., Total Enrollment). Do the same for your color legend (e.g., Poverty Rate (%)).
11. Click the **Tooltip card**. Delete the default text in the editor window. Use the Insert dropdown menu in the top-right corner to add your loaded fields, formatting the text to read cleanly:
    County: `<County Name> | Enrollment: <SUM(total_enrollment)> | Poverty: <SUM(poverty_rate)>%`
12. Verify that your state's counties display circles of varying sizes, the color ramp accurately reflects the data range, and the tooltip displays the correct county information upon hover.



Dual-Axis (Layered) Map

A dual-axis layered map overlays two map types on one canvas. This combo allows for simultaneous viewing of regional patterns and local magnitudes. It's helpful for assessing health outcomes in relation to problem sizes, for example, by shading counties based on diabetes rates and adding symbols for health professional shortages over a social vulnerability index.

Data Structure Needed

Two layers are combined in Tableau: Layer 1 (choropleth) uses `fips` and any rate variable (such as `diabetes_pct`), while Layer 2 (symbols) uses `fips` and a count variable.

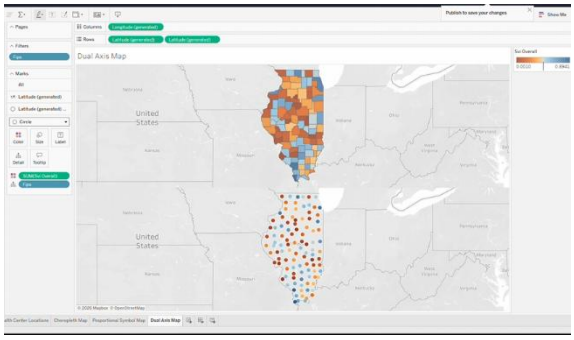
```
Layer 1 (choropleth base):
fips      diabetes_pct  svi_overall
17001        11.2          0.62
Source: sdoh_il_tableau.csv

Layer 2 (symbols):
name          latitude(generated)   longitude(generated)  total_enrollment
Adams County     39.9217                    -91.1849             350
Source: sdoh_il_tableau.csv
```

Step-by-step: Dual-Axis (Layered) Map
To build this combination, follow these exact steps:

Part 1: Build the Background Layer

1. Click your sheet tab at the bottom and rename it: Dual Axis Map.
2. Right-click FIPS in the Data pane, hover over **Geographic Role**, and select **County**
3. Double-click the FIPS field to generate the base map layout.
4. Drag FIPS from the Data pane and drop it onto the **Filters** shelf. Click the **Condition** tab, select **By formula**, and enter: `LEFT([Fips], 2) = '17'` for Illinois. Click OK.
5. Change the mark type dropdown on the **Marks card** from *Automatic* to **Map** to create your solid filled counties.
6. Drag `svi_overall` from the Data pane and drop it directly onto the **Color** shelf.
7. Click the **Color card**, select **Edit Colors**, and choose a diverging palette like **Orange-Blue Diverging** (since SVI is an index variable). Click OK.

Part 2: Layer the Circles on Top



8. Hold the `Ctrl` key on your keyboard, click the green **Latitude (generated)** pill on the Rows shelf, and drag it to the right to duplicate it. Your screen will split into a top and bottom map.
9. Click the **Latitude (generated) (2)** section on your **Marks** card to open the controls for your second layer.
10. Change the mark type dropdown for this second layer from *Map* to **Circle**.
11. Drag `svi_overall` off this second Marks card to clear it out.
12. Drag `diabetes_pct` from the Data pane and drop it onto the Size shelf. The circles will now scale based on diabetes prevalence.
13. Drag `County Name` from the Data pane and drop it onto the **Detail** tile to load it into tooltip memory.
14. Go back to the **Rows** shelf at the top, right-click the second green **Latitude (generated)** pill, and select **Dual Axis** to merge the dots onto your filled background.
15. Click the **Color** tile on your circle **Marks** card and set the **Opacity to 70%** so you can see the SVI colors underneath your diabetes circles.

Filled Map with Pie Charts
A map overlays pie charts on county centroids over a choropleth map to show categorical breakdowns across regions. It reveals both overall rates and internal composition at once but should be used for only two to three categories, as smaller pie charts become hard to read. In public health, it helps visualize

resource or burden distribution within counties, like health shortage types or other categorical metrics across counties.

Data preparation in R (required before building in Tableau)
Your master CSV has columns for `hpsa_prim_care`, `hpsa_dental`, and `hpsa_mental_health`. Reshape it to long format before importing into Tableau.

```
# Data preparation in R (required before building in Tableau)
# Run this in R after completing Part 2 Scripts #1-#6

library(tidyverse)

# ─── Load master CSV ──────────────────────────────────────────────────
STATE_LABEL <- "il"   # change to match your state
master <- read_csv(paste0("data/master/sdoh_", STATE_LABEL, "_master.csv"),
                   col_types = cols(fips = col_character()))

# ─── Reshape HPSA columns to long format for Tableau Pie Chart Map ────────
# Note: "count" stores the HPSA designation level:
# 0 = none, 1 = partial, 2 = full
hpsa_long <- master %>%
  select(fips, county_name, hpsa_prim_care, hpsa_dental, hpsa_mental_health) %>%
  pivot_longer(
    cols = c(hpsa_prim_care, hpsa_dental, hpsa_mental_health),
    names_to  = "shortage_type",
    values_to = "count"
  ) %>%
  mutate(shortage_type = case_when(
    shortage_type == "hpsa_prim_care"    ~ "Primary Care",
    shortage_type == "hpsa_dental"       ~ "Dental",
    shortage_type == "hpsa_mental_health"~ "Mental Health"
  ))

cat("Rows in reshaped file:", nrow(hpsa_long), "\n")
cat("Expected:", nrow(master) * 3, "(3 shortage types x", nrow(master),
"counties)\n")

write_csv(hpsa_long, "output/tableau/hpsa_long.csv")
cat("Saved: output/tableau/hpsa_long.csv\n")
```

Note. count represents the HPSA designation level

Data Structure Needed
Long-format data with one row per county per category:

```
fips         shortage_type         count
17001         Primary Care           2
17001         Dental                 1
17001         Mental Health          1
17003         Primary Care           3
17003         Mental Health          2
...               ...               ...

Required: fips (String), category column, count
Format: long (one row per county-category pair)
Create in R: hpsa_clean %>% pivot_longer(...)
```

Step-by-step: Filled Map with Pie Charts Using Two Datasets
Part 1: Connect the Two Datasets

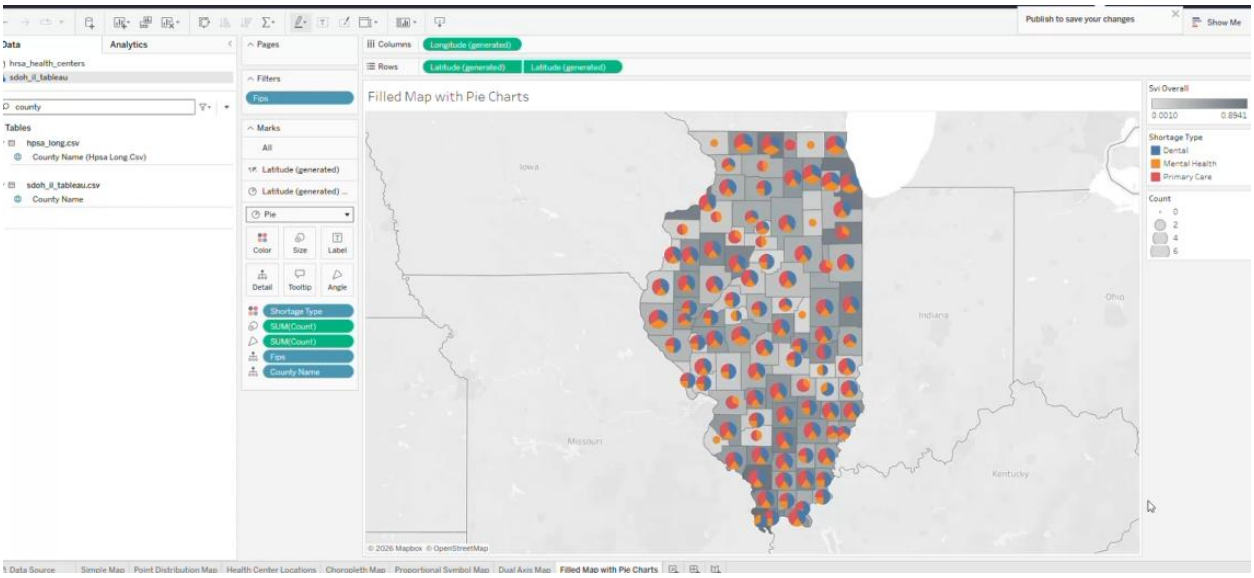
1. Click the **Data** menu, select **New Data Source**, click **Upload from Computer**, and upload `sdoh_[state]_tableau.csv`.
2. Click the small **+ (Add)** icon next to the **Connections** header. Select **Upload from Computer** and upload `hpsa_long.csv`.
3. Drag the `hpsa_long.csv` file from the left panel and drop it to the right of the `sdoh_[state]_tableau.csv` box.
4. Set the matching fields in the relationship mapping box that pops up at the bottom:
    - Under `sdoh_[state]_tableau`, select **Fips**.
    - Under `hpsa_long.csv`, select **Fips**.
5. Click the **Sheet** tab in the bottom-left corner to open your workspace canvas.

Part 2: Build the Background Map Layer

6. Click the arrow next to `sdoh_[STATE]_tableau.csv` to expand it. Ensure that all variables for this background layer are sourced from your master CSV file.
7. Click `Fips` in the Data pane, change data type to **String**, hover over **Geographic Role**, and select **County**.
8. Double-click the `Fips` field to automatically generate your baseline map layout using Tableau's built-in coordinates.
9. Drag `Fips` from the Data pane onto the **Filters** shelf. Click the **Condition** tab, select **By formula**, and enter: `LEFT(STR([Fips]), 2) = '17'` to cleanly isolate Illinois. Click **OK**.
10. Change the mark type dropdown menu on the Marks card from *Automatic* to **Map** to create solid filled county shapes.
11. Drag `Svi Overall` from the Data pane and drop it onto the **Color** shelf of the Marks card. Click **Color → Edit Colors**, select the **Gray** palette

Part 3: Layer the Pie Charts Using a Dual Axis

12. Hold the `Ctrl` key on your keyboard, click the green **Latitude (generated)** pill sitting on your **Rows** shelf, and drag it to the right to duplicate it. Your screen will split into a top and bottom map.
13. Click the lower section of your split Marks card labeled **Latitude (generated) (2)** to isolate the settings for your second layer.
14. Drag the green `Svi Overall` pill off this second Marks card to clear background shading from your charts.
15. Change the mark type dropdown on this second Marks card from *Map* to **Pie**.
16. Expand `hpsa_long.csv`. All variables for this layer come from this dataset.
17. Drag **Shortage Type** from the Data pane and drop it onto the **Color** tile of this Marks card to define your individual pie slices.
18. Drag **Count** onto the **Angle** to control slice size. Drag **Count** onto the **Size** to scale the pie diameter by the shortage volume.
19. Go back up to your top **Rows** shelf, right-click the second green **Latitude (generated)** pill, and click **Dual Axis** to snap your pie charts directly over your shaded county background map.
20. Click the **Size** on the pie Marks card and adjust the slider until the charts are visible without overlapping borders. Drag `County Name` to Detail.
21. Click the **Tooltip** card on the pie Marks card, delete the default text, and use the **Insert** dropdown menu to format your clean label:
    County: <County Name> | Shortage Type: <Shortage Type> | Designation Count: <Count>



5. Assemble Your Dashboard

Corresponding Video: [All Maps (Thematic Maps)]

A Dashboard combines multiple coordinated views, any combination of the maps you built in the previous sections, into one interactive canvas. Users can filter one view, and all others update automatically.

Choose Which Views to Include

Your dashboard must include at least three views. Choose from any maps you built in the previous sections. The recommended combination for SDOH research is:

- View 1 (primary): Filled (Choropleth) Map of your outcome variable, showing WHERE the burden is highest.
- View 2 (supporting): Ranked Bar Chart of your outcome variable, showing WHICH counties rank worst.
- View 3 (supporting): Scatter Plot of predictor vs outcome, showing the SDOH relationship.

You can substitute or add any map. For example, replace the bar chart with a Point Distribution Map showing HPSA shortage locations or add a Proportional Symbol Map as a fourth view.

Build Your Ranked Bar Chart

The ranked bar chart shows every county sorted from highest to lowest by your outcome variable.

1. Upload `sdoh_[state]_tableau.csv`. Click your sheet tab at the bottom and rename it: Bar Chart.
2. Drag `County Name` from the Data pane to the Columns shelf, then drag `[OUTCOME_Variable]` from the Data pane to the Rows shelf to create your vertical bars.
3. Sort the chart in descending order by clicking the **Sort Descending** icon in the top toolbar (the button showing vertical bars with a down arrow).
4. Click the **Analytics** tab next to the Data pane. Drag **Average Line** onto the canvas and drop it directly onto the **Table** box that appears. Right-click the newly created line, select **Edit**, and change the **Label** dropdown to **Custom**, typing: `State Average: <Value>`.

5. Drag `[PREDICTOR_Variable]` to the **Color** on the Marks card. Darker bars will now represent higher predictor values.

Build Your Scatter Plot

The scatter plot shows the relationship between your predictor variable on the X axis and your outcome variable on the Y axis where each dot represents an individual county.

1. Click your sheet tab at the bottom and rename it: Scatter Plot.
2. Drag `[PREDICTOR_Variable]` to the **Columns** shelf and `[OUTCOME_Variable]` to the **Rows** shelf.
3. Click **Analysis** in the top menu, then **uncheck Aggregate Measures**.
4. Click the **Analytics** tab, drag **Trend Line** to canvas, then drop it into the **Linear** box.
5. Right-click your **X-axis scale**, select **Edit Axis...,** and set the **Axis Title** to your predictor's label. Repeat for the Y-axis.

Dashboard Layout Design

Dashboard layout guides the reader's eye and clarifies view relationships. A poorly designed dashboard feels overwhelming, even with good data. Here are principles used by professional SDOH dashboard designers.

Principle (1) Lead with the map, support with charts

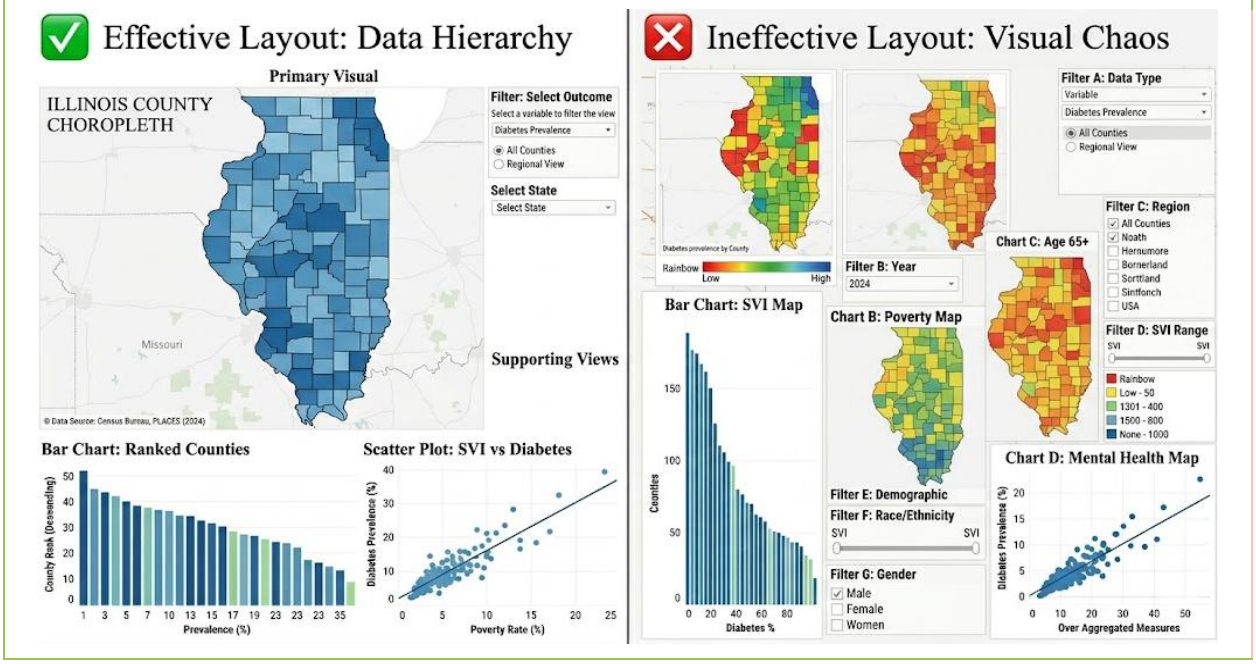
The choropleth map shows "where," being the most visually compelling element, usually in the largest area on top or left. Supporting charts (bar, scatter) indicate "how much" and "what relationship," placed smaller below or to the right.

✓ **Effective layout**

- Large choropleth map in the top or left position
- Bar chart below or to the right
- Scatter plot below or to the right
- Filter panel narrow, on the right edge
- Title and source note at top and bottom

✗ **Ineffective layout**

- Four charts all the same size competing equally
- Map squeezed into a small corner
- Bar chart taller than the map
- Filters scattered across the canvas
- No title, no data source note



Principle (2) Use filter actions, not separate dashboards
Filter actions let users click one view to update all others simultaneously. Clicking a geographic unit on the map highlights the same unit in the bar chart and scatter plot. Design your dashboard so each view is linked to all others. Instructions for adding filter actions are in the Assemble the Dashboard Canvas section below.

Principle (3) One variable per map, multiple variables across views
A choropleth map shows a single variable using color. Adding a second variable with size or shape causes visual overload. Instead, show one variable per map and use a scatter plot to display the relationship between two variables.

Dashboard contains: Map: diabetes_pct (color only) Scatter: poverty_rate (X) vs diabetes_pct (Y) Bar: counties ranked by diabetes_pct ✓ Each view has one job. They answer: where is diabetes most prevalent, which counties rank highest, and is poverty associated with it?	Dashboard with variables on one map using color, size, and shape: Map: diabetes_pct (color), poverty (size), and SVI (shape) ✗ Visually overwhelming. The reader cannot accurately perceive three visual channels at once. Each variable competes for attention.

Principle (4) Size communicates priority
In Tableau, the size of each view on the dashboard canvas signals its importance to the reader. Use size deliberately:

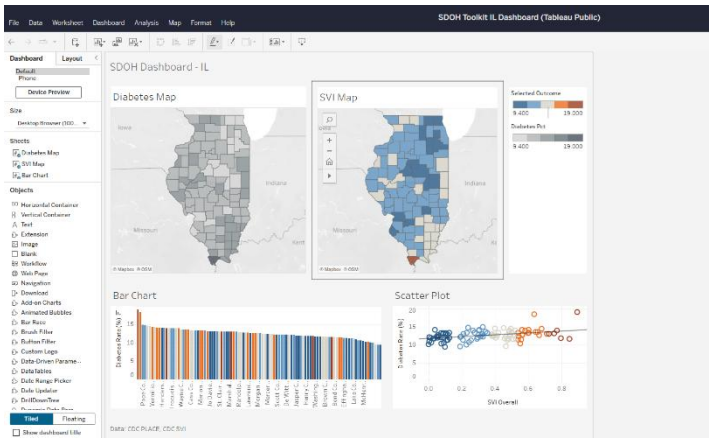
- Largest view (50–60% of canvas): Your primary finding is usually shown on the choropleth map, which you want the reader to see first.
- Medium views (20–25% each): Supporting context. Charts and plots illustrating map patterns.
- Narrow strip or small panel: Filter controls, legend, parameter selectors. Keep them small.
- Full-width banners (very narrow): Title text and data source note. It needs to be easy to find when readers look for it.

Principle (5) Title every view, cite every data source
Every sheet on the dashboard needs its own title explaining what it shows. The dashboard itself needs a source note at the bottom. Untitled views force the reader to guess what they are looking at.

✓ **Good sheet title format** • Choropleth map: "Diabetes Prevalence by County (%)" • Bar chart: "Illinois Counties Ranked by Diabetes Prevalence" • Scatter: "Poverty Rate vs. Diabetes Prevalence, Illinois Counties" • Source note: "ACS 2020–2024 · CDC PLACES 2025 · SVI 2022"	✗ **Poor sheet title format** • "Map" or "Sheet 1" • "Chart" or no title at all • "Scatter Plot" • No data source note anywhere on the dashboard

Corresponding Video: Assemble Your Dashboard

Assemble the Dashboard Canvas



1. Click the **New Dashboard** icon at the bottom of the screen.
2. Drag a **Vertical Container** from the left panel onto the empty canvas.
3. Drop your **map** sheet into the top of the canvas and drag its bottom edge until it occupies 55% to 60% of the dashboard height.
4. Drag a **Horizontal Container** from the left panel and drop it directly underneath the map.
5. Drop your **Bar Chart** into the left side of the horizontal container and your **Scatter Plot** into the right side.
6. Drag a **Text** box to the very top of the dashboard and type your title using the formatting from Principle 5 at 16pt and regular weight.
7. Drag a **Text** box to the very bottom of the dashboard and list your active public health datasets in 10pt gray font.
8. Select **Dashboard** from the top menu, click **Actions**, select **Add Action**, choose **Filter**, and set the run action option to **Select** to link your map, bar chart, and scatter plot.
9. Click a county on the map to confirm that both the bar chart and scatter plot instantly filter to isolate that same area, then click a blank space to reset the view.

Save and Publish to Tableau Public

1. Click the blue **Publish As** button in the top right corner of the screen to open the save dialog window.
2. Type **SDOH-Toolkit-[State]-Dashboard** in the workbook name field and click the blue **Publish** button.
3. Wait for the browser to load your newly published dashboard page on the Tableau Public website.
4. Close the editing window. The window will show the Tableau Public page.
5. Click the **Share** icon in the top-right corner to open distribution settings. Copy the URL from the **Link** box for direct sharing. Copy the HTML code from **Embed Code** to embed the dashboard into a website or report.

6. Build Your Story Map

Corresponding Video: Story Map

A Tableau Story uses the built-in story feature to arrange sheets and dashboards into a guided narrative. Unlike a Dashboard, which shows all views at once, a Story displays one view with annotations, guiding readers through your research, data, and findings step by step.

Story Map vs. Dashboard

Dashboard	Story Map
Shows multiple views at once	Shows one view at a time in guided sequence
Users explore with filter actions	Author guides users through a specific argument
Best for: data exploration, public reporting	Best for: presentations, policy briefs
Built from any map views in Sections 3 and 4	Built from same sheets

Choose Which Views to Include in Your Story
A story can include any sheet or dashboard you have already built. A recommended five point narrative for socio-spatial health research includes the following structure.

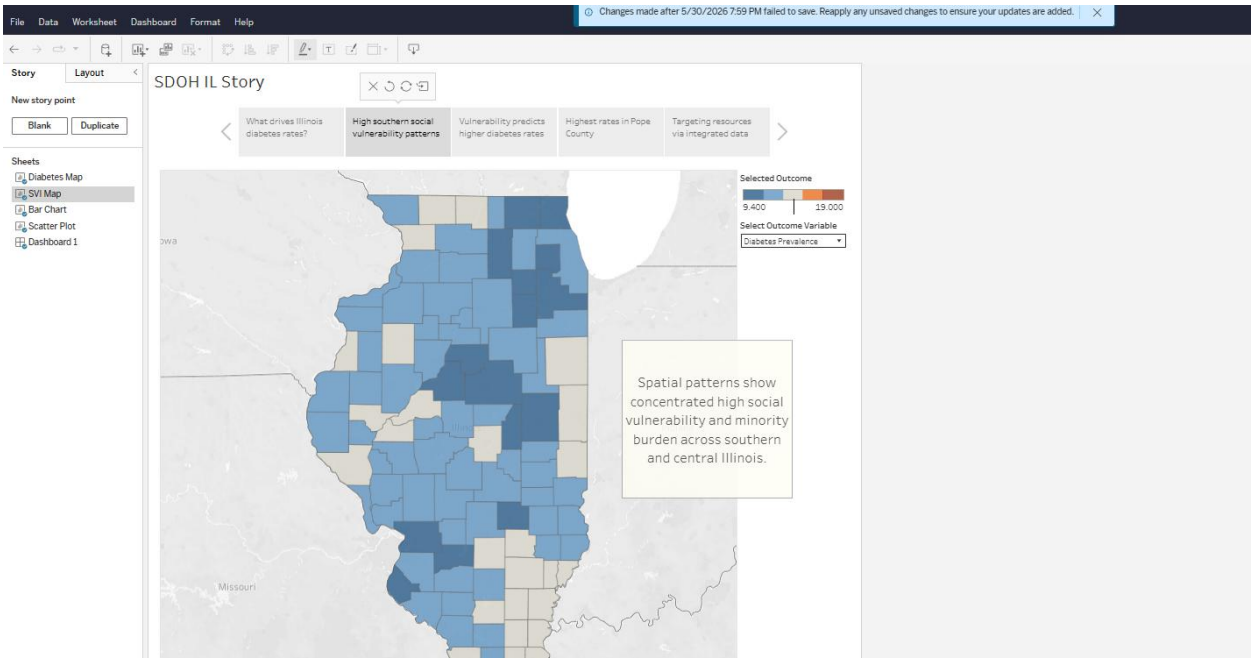
- Story point 1. Research question to show your outcome variable map.
- Story point 2. Strongest predictor map to illustrate social vulnerability.
- Story point 3. Scatter plot with a trend line to show their mathematical relationship.
- Story point 4. A ranked bar chart to demonstrate specific county-level disparities.
- Story point 5. Completed a full dashboard to give a holistic, interactive view.

Step-by-step: Build Your Story

1. Click the **New Story** icon at the bottom of the screen to open the story workspace canvas.
2. Build each of your five story points using the layout matrix below, clicking the **Blank** button in the left panel to generate each new slide.

Story Point	Caption	Map/Figure Sheet	Text Box
Story Point 1. Research question	What drives Illinois diabetes rates?	Drag your outcome Map sheet from the left pane.	
Story point 2. Predictors	High southern social vulnerability patterns	Drag your Predictor Map sheet from the left pane.	Spatial patterns show concentrated high social vulnerability and minority burden across southern and central Illinois.
Story point 3. The relationship	Vulnerability predicts higher diabetes rates	Drag your Scatter Plot sheet from the left pane.	Linear regression shows a positive link: higher social vulnerability predicts higher diabetes rates, supported by the upward trend.
Story point 4. Geographic patterns	Highest rates in Pope County	Drag your Bar Chart sheet from the left pane.	Geographic differences are stark, with southern rural areas like Pope County showing the highest diabetes rates at over 15 percent.
Story point 5. Conclusion	Targeting resources via integrated data	Drag your full SDOH Dashboard-IL from the left pane.	This dashboard enables public health practitioners to cross-reference geographic disease burdens with social vulnerability, optimizing resource allocation and intervention targeting.

3. Click through each narrative tab at the top of the story grid to verify that all five points display their respective caption text, visualizations, and text boxes properly.
4. Click the blue **Publish** button in the top right corner of the screen to save your complete data story online.



7. Adapt and Share

Corresponding Video: [Adapt and Share]

Change Variables on an Existing Map
Instead of building a brand-new sheet from scratch, you can quickly swap out the data on your current map or add a dynamic dropdown menu for your users.

Method A: Swap directly (fastest)

- Open your map sheet. In the Marks card, find the green pill on the Color shelf.
- From the Measures list, drag your new variable directly onto the existing green pill. Tableau swaps the variable and re-shades the map immediately.
- Update the color palette, tooltip, and sheet title to match the new variable.

Method B: Add a parameter (lets users switch variables)

1. Click the small drop-down arrow next to the search bar in the Data pane and select **Create Parameter**.
2. Name the parameter `Select Outcome Variable` and change the **Data type** dropdown to **String**. Select **List** under the **Allowable values options** to open the choices grid.
3. Type your exact raw database variable name (e.g., Diabetes Pct) in the **Value column** and type your label (e.g., Diabetes Prevalence) in the **Display As column**.
4. Type your second raw database variable name (e.g., Depression Pct) in the next row under the **Value** column and type your label (e.g., Depression Prevalence) in the **Display As** column before clicking OK.
5. Click the drop-down arrow next to the search bar again and select **Create Calculated Field**. Name the calculation, `Selected Outcome`, and type the matching logic formula:

```
IF [Select Outcome Variable] = 'Diabetes Pct' THEN [Diabetes Pct]
ELSEIF [Select Outcome Variable] = 'Depression Pct' THEN [Depression Pct]
```

```
ELSE NULL
END
```

6. Drag your new **Selected Outcome** calculated field from the Data pane and drop it directly on top of the existing color variable on your Marks card to replace it.
7. Hover over **"Select Outcome Variable"** under the Parameter at the bottom of the Data pane. Click the down arrow, select **Show Parameter** to add the interactive dropdown menu to your workspace.

Export Maps for Reports or Presentation/Publication

Method A: Download from Tableau Public	Method B: Browser Screenshot

• Navigate to your final published dashboard link on the Tableau Public website.
• Click the **Download** icon located in the upper right toolbar area of the published webpage layout.
  ○ Select **Image** from the format options menu to save a flat picture file.
  ○ Select **Workbook** instead if you want to save the raw **twbx** pack file to your computer.
	• Press the Windows key + Shift + S on a Windows keyboard to open the snipping menu.
	• Press the Command key + Shift + 4 on a Mac keyboard to activate the crosshair crop utility.
	• Drag your cursor selection box to isolate just the active visualization frame while completely leaving out the outer web browser address bars.

Adapt the Dashboard for Another State

Because you enabled downloads, any student can adapt your dashboard.

```
Step 1: Go to the published dashboard. Click the download icon. Select Tableau
Workbook.
Step 2: Go to public.tableau.com. Sign in. Click Create a Viz.
Step 3: Upload from Computer. Upload the .twbx file you downloaded.
Step 4: Confirm FIPS is String (Abc) in the Data Source pane.
Step 5: Update the FIPS filter. Change 'starts with 17' to your state prefix.
        (Indiana = 18, Wisconsin = 55, Missouri = 29, Michigan = 26)
Step 6: Update dashboard titles to reflect your state and variables.
Step 7: File > Save to publish under your own Tableau Public profile.
```

Troubleshooting Common Tableau Problems

These are the most common issues students encounter when building SDOH maps in Tableau, with specific fixes.

Problem	What you see	Fix
Map does not render / shows blank	Nothing appears, or a world map displays without any geographic units.	Go to Data Source tab. Confirm FIPS shows Abc (string variable). Right-click FIPS, then Geographic Role, then County. Return to the sheet and double-click FIPS again.
Only some counties appear (others gray)	Most counties are shaded correctly, but some are displayed in gray.	Gray means missing data for that county. Run filter(master, is.na(your_variable)) in R to identify which geographic units have missing values.
Map shows entire U.S. not just your state	All 3,000+ U.S. counties appear	Edit the FIPS filter. Click the F**ilter → Edit Filter → Condition, select** By formula and type: LEFT([Fips],2) = '17'. Use straight quotes, not curly quotes.

Bar chart shows county codes not names	Bars labeled 17001, 17003 instead of county names	Remove FIPS from Rows. Drag County Name (the text column) to Rows instead.
Scatter plot shows one big dot	All geographic units collapse to a single point	In the **Analysis** menu, uncheck Aggregate Measures. Each geographic unit should now appear as a separate dot.
Filter actions not working	Clicking a geographic unit does not update other views	Dashboard, then Actions. Check that source and target sheet names match exactly. Sheet names are case-sensitive.
Trend line p-value differs from Script #7	Tableau shows different p-value than your R regression	Tableau uses all available rows. Script #7 used `drop_na()`. Note: The Tableau trend line includes all geographic units; the regression model used listwise deletion.
Point map for rate data	Each area has a dot at its center instead of shading, implying that the data applies to a single point.	Right-click FIPS → Geographic Role → County. In Marks card, confirm mark type is Map.
Choropleth map for facility locations	The entire geographic area is shaded to indicate that one clinic is present. This implies the entire area has that property.	For facility data (lat/lon CSV), drag Longitude to Columns, Latitude to Rows. Marks should select Circle, not Map.

8. Make Your Project Citable, Reproducible, and Findable

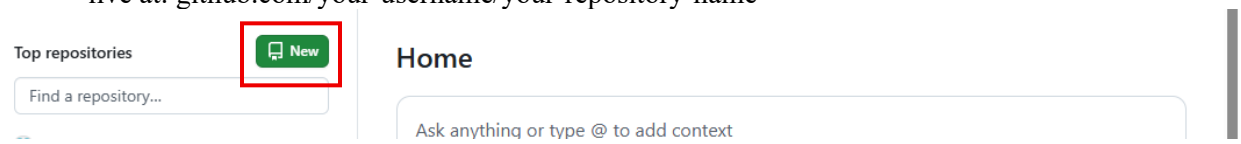
GitHub is a free platform for hosting, sharing, and version control of your research projects. Uploading your code, documentation, and shareable project files to GitHub supports reproducibility. Before beginning this section, you should have your completed project folder from Part 2, which should include R scripts, the master CSV, data dictionary, and README.txt.

Create a Free GitHub Account

1. Go to github.com and click Sign up in the top right corner.
2. On the registration page, click the **Continue with Google** button to link your Gmail account. Choose a professional GitHub username since this handle will be visible on your CV, resume, and LinkedIn profile (for example, hpotter-uis or harry-potter).
3. Complete the brief onboarding steps, ensuring you select the Free tier if prompted, and your new account is ready to go.

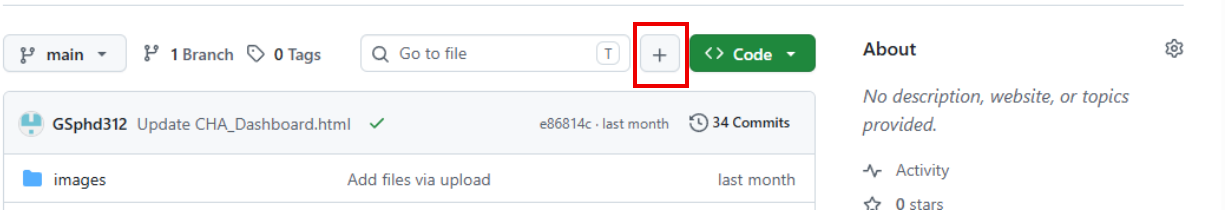
Create a New Repository

1. A repository (repo) is a folder on GitHub that holds all your project files. Click the **New** icon in the top right corner of GitHub to create a new repository.
2. Fill in the repository details with the repository name in lowercase (no spaces) and check the **Choose visibility*** set to Public. Then, click **Create repository.** Your empty repository is now live at: github.com/your-username/your-repository-name



Upload Your Project Files

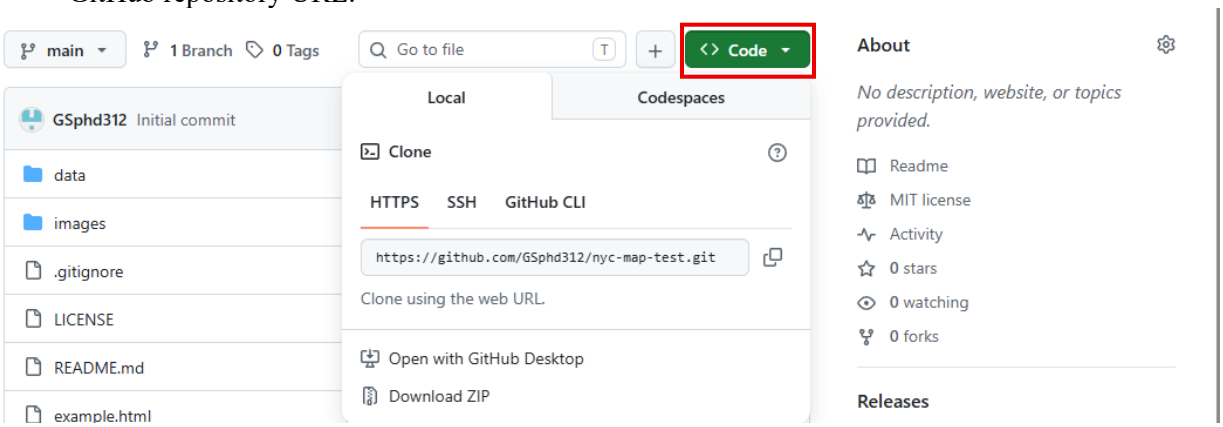
1. On your repository page, click **+**, then select Upload files.



2. From your project folder, select and drag the following files: all R scripts from your code/ folder, your master CSV, your Tableau CSV, your data dictionary, and your README.txt. Then, click the **Commit changes** button. All your files are now live in your public repository.

Copy Your Repository URL

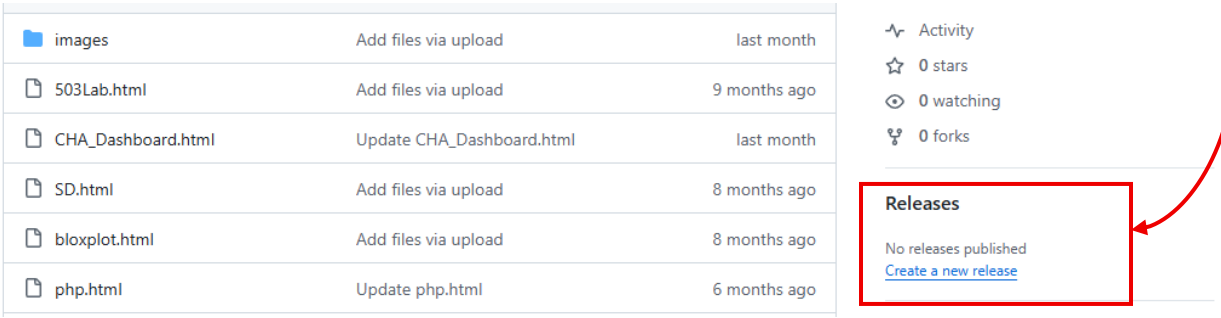
1. Click the green **< > Code** button on your repository page and copy the URL shown. This is your GitHub repository URL.

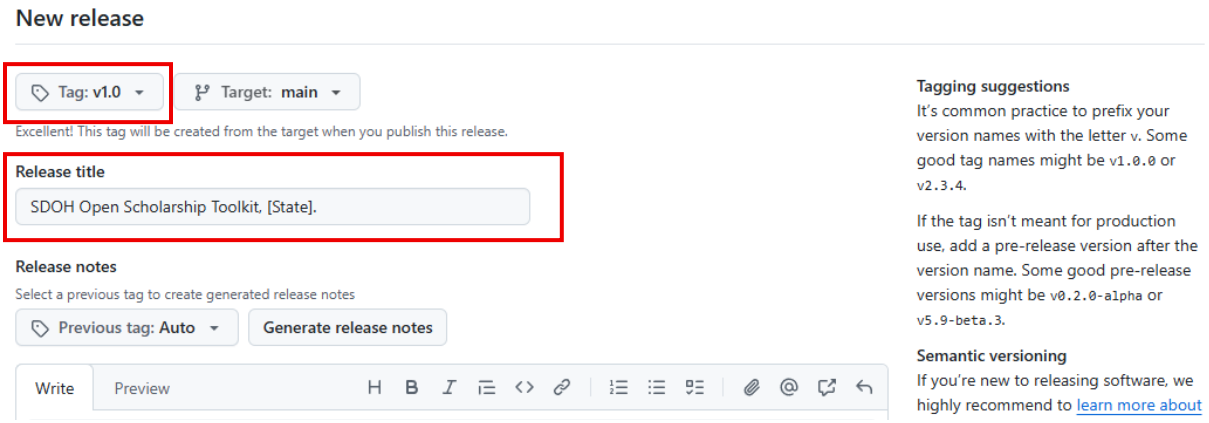


2. Paste it into your README.txt under the GitHub repository URL.

Register a DOI with Zenodo

1. Go to zenodo.org and click **Log in with GitHub** to connect your accounts automatically.
2. Once logged in, click your username in the top right corner of Zenodo and select **GitHub** from the dropdown menu. Hit the **Sync now** button to update the list from GitHub. Then, find your toolkit repository in the list and toggle it to **ON**.
3. Back in GitHub, click **Releases** on the right side of the repository page, then click **Create a new release**. Click **Tag: Select tag**, then select **Create new tag**. Type v1.0, set the **Release title** to SDOH Open Scholarship Toolkit, [State]. Click **Publish release**.





4. Zenodo archives the release and creates a citable record. Processing time may vary. It will look like https://doi.org/10.5281/zenodo.XXXXXXX.
5. Copy your DOI link and add it in two places: at the top of your README.txt file and in the description of your Tableau Public dashboard.

Quick Conceptual Video: How to Make Your GitHub Code Citable with Zenodo | pyOpenSci. The button layouts on screen may vary slightly from the current live site.

Institutional Repository Sharing (Optional)

An institutional repository provides a university-supported location for preserving and sharing scholarly work. IDEALS (Illinois Digital Environment for Access to Learning and Scholarship) is the University of Illinois institutional repository and is one example. Learners affiliated with other institutions may use their own institutional repository or another appropriate open repository.

- For UIS-affiliated learners, submitting your project to IDEALS adds it to the UIS Collection of research, making it permanently accessible and linkable from your resume and LinkedIn.
- Unlike Zenodo, IDEALS does not assign a DOI but provides a permanent URL hosted by the university.
- To submit, complete the UIS form. The UIS library will follow up when your submission is available or if they have questions. For more info, contact the UIS Library.

Side-by-Side Comparison of IDEALS and Zenodo

	IDEALS	**Zenodo**
What it is	UIS institutional repository	International Open Research Repository (CERN)
Who runs it	University of Illinois	CERN and OpenAIRE
DOI	No, permanent URL only	Yes, DOI is assigned automatically
What you deposit	Presentations/posters/reports	Datasets, code, any research output
How to submit	UIS form with librarian reviews and uploads.	Self-service
Best for	Presentations/posters/reports	Datasets, R scripts, GitHub repositories
UIS connection	Official UIS collection, institutional identity	No UIS branding
Voluntary	Yes	Yes

Your project is now fully open and FAIR:

FAIR	**What You Did**	**Where in This Toolkit**	**Open Scholarship Requirement**

Findable	GitHub README, Zenodo DOI, and IDEALS permanent URL	Part 3: GitHub, Zenodo IDEALS	Share your work openly
Accessible	Tableau Public dashboard, public GitHub repository, CSV format	Part 3: Tableau Public & GitHub	Share your work openly
Interoperable	FIPS codes for merging, standard column names, open file formats	Part 1 and Part 2	Make it reproducible
Reusable	Commented R scripts, codebook, data citations with download dates	Part 2	Cite your data and make it reproducible

9. Part 3 Checklists

- ☐ Check every item before submitting your Tableau Public URL. Complete all applicable items before submitting or sharing your Tableau Public URL is: public.tableau.com/views/_____
- ☐ I have a Tableau Public account and am signed in at public.tableau.com.
- ☐ My master CSV (sdoh_[state]_tableau.csv) is uploaded and FIPS is set to String (Abc).
- ☐ I have built at least one Asset Map view (Simple Map or Point Distribution Map).
- ☐ I have built at least one Thematic Map view. The Filled (Choropleth) Map is required. Proportional Symbol, Dual-Axis, and Filled Map with Pie Charts are optional.
- ☐ I have assembled a Dashboard with at least three coordinated views.
- ☐ My choropleth map uses the correct color palette (sequential or diverging — not rainbow).
- ☐ Filter actions are working — clicking the map updates the bar chart and scatter plot.
- ☐ Every sheet has a descriptive title (not Sheet 1 or Map).
- ☐ The dashboard has a data source note at the bottom listing all datasets used.
- ☐ The tooltip on each view shows geographic unit name, outcome value, and at least one predictor value.
- ☐ I have built a Story Map with at least three story points linking my research question, findings, and conclusion.
- ☐ The dashboard is published to Tableau Public and set to Public visibility.
- ☐ Workbook downloads are enabled so others can adapt the template.
- ☐ The Tableau Public URL is recorded and ready to share.
- ☐ My GitHub repository contains R scripts, master CSV, data dictionary, and README.txt. If required for a course, I have submitted the repository URL according to my instructor's directions.
- ☐ When appropriate for my project, I have created a citable Zenodo record and added the DOI or persistent link to my README and Tableau Public dashboard description.
- ☐ If appropriate, I have deposited or shared my work through an institutional or open repository.

✓ Your dashboard is published, publicly accessible, and documented for reuse. That is open scholarship.

Part 1: SDOH Data Orientation · Part 2: Download & R Workflow · Part 3: Tableau, Repository & Open Sharing